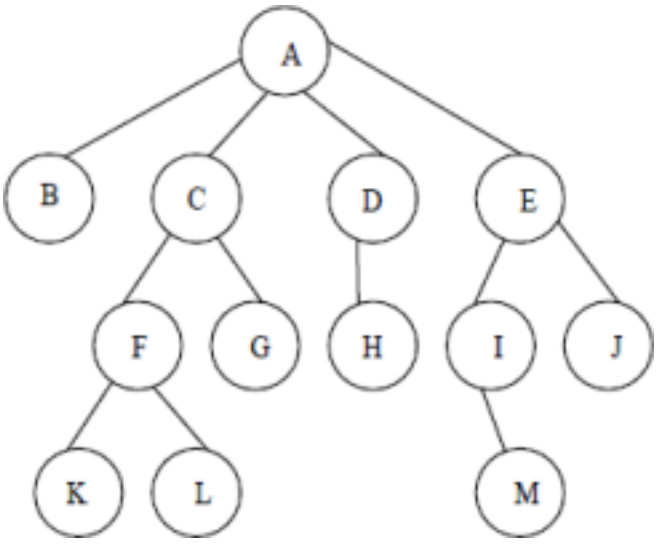
UNIT II

TREES AND THEIR APPLICATIONS

Trees - Tree Terminology – Binary Trees —Expression Tree - Binary Search Trees– AVL Trees- - Tree Traversals-Priority Queues (Binary Heap). - Trie (prefix tree) and real-time use in autocomplete and spell check, Segment Tree / Binary Indexed Tree (introductory level), Applications in databases, compilers, dictionaries, Real-world problems: Spell checkers, auto-suggestion engines, Huffman coding.

2.1 TREE

A tree is a finite set of one or more nodes such that there is a specially designated node called the Root, and zero or more non empty sub trees T1, T2, … Tk, each of whose roots are connected by a directed edge from Root R.



2.2 TREE TERMINOLOGY

NODE : Item of Information.

ROOT : A node which does not have a parent. Here, Root is A.

LEAF : A node which does not have children is called leaf or Terminal node. Here B, K, L, G, H, M, J are leafs.

SIBLINGS : Children of the same parents are said to be siblings, Here B, C, D, E are siblings, F, G are siblings. Similarly I, J & K, L are siblings.

PATH : A path from node n1 to nk is defined as a sequence of nodes n1, n2,n3 . ..... nk such that ni is the parent of ni+1. for 1 = i to k . There is exactly only one path from each node to root.

Here, path from A to L is A, C, F, L. where A is the parent for C, C is the parent of F and F is the parent of L.

LENGTH : The length is defined as the number of edges on the path. Here, the

length for the path A to L is 3.

DEGREE : The number of sub trees of a node is called its degree. Here Degree of A is 4 Degree of C is 2

The degree of the tree is the maximum degree of any node in the tree. Here, the degree of the tree is 4.

LEVEL : The level of a node is defined by initially letting the root be at level one, if a node is at level L then its children are at level L + 1. Level of A is 1. Level of B, C, D, is 2. Level of F, G, H, I, J is 3 Level of K, L, M is 4.

DEPTH : For any node n, the depth of n is the length of the unique path from root to n. The depth of the root is zero. Here, Depth of node F is 2. Depth of node L is 3. **HEIGHT :** For any node n, the height of the node n is the length of the longest path from n to the leaf. The height of the leaf is zero Here, Height of node F is 1. Height of L is 0.

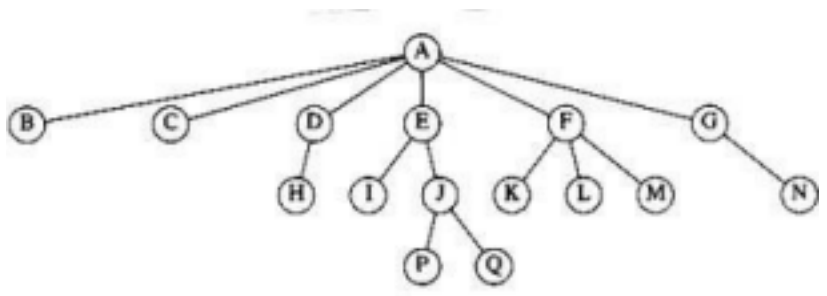
- The height of the tree is equal to the height of the root
- Depth of the tree is equal to the height of the tree.

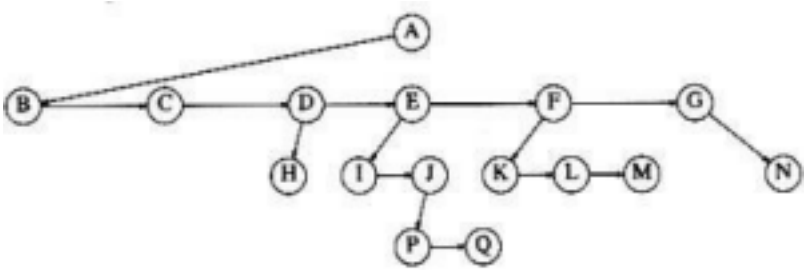
Implementation of Trees

One way to implement a tree is creating a node and in each node, store data, a pointer to each child of the node. Keep the children of each node in a linked list of tree nodes.

```
class Node:
 def __init__(self, data):
 self.data = data
 self.left = None # Pointer to the left child node
 self.right = None # Pointer to the right child node
```

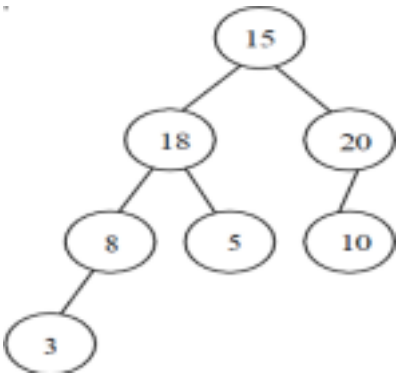
First child/next sibling representation of the tree





2.3 BINARY TREE

• Binary Tree is a tree in which no node can have more than two children. • Maximum number of nodes in the tree with level i is 2^i-1. (Assume root node is at level 1)

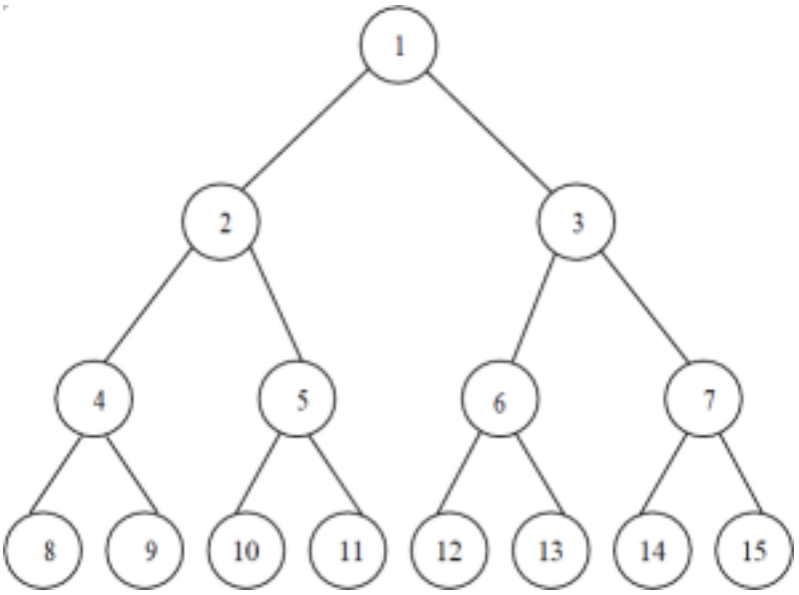


Binary Tree Node Declarations

```
class Node:
 def __init__(self, value):
 self.value = value # The data stored in the node
 self.left = None # Reference to the left child node
 self.right = None # Reference to the right child node
```

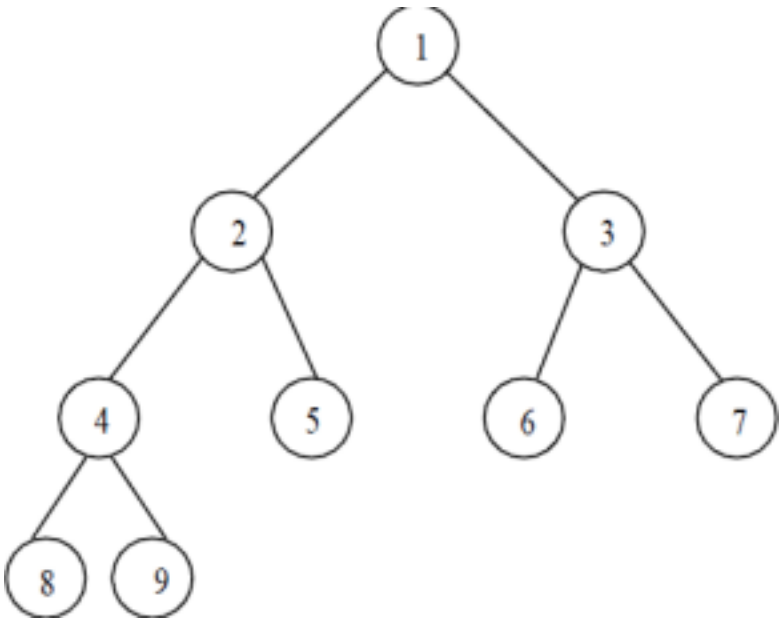
Full Binary Tree

- A full binary tree of height h has 2^{h+1}- 1 nodes.
- Here height is 3.
- No. of nodes in full binary tree is = 2^{3+1}-1= 15 nodes.
- A full binary tree (sometimes proper binary tree or 2-tree) is a tree in which every node other than the leaves has two children.



Complete Binary Tree:

• A binary tree is said to be a complete binary tree when all the levels are completely filled except the last level, which is filled from the left • A complete binary tree of height h has between 2h and $^{2h+1}$- 1 nodes. • In the bottom level the elements should be filled from left to right.



• A full binary tree can be a complete binary tree, but all complete binary tree is not a full binary tree.

Representation of a Binary Tree

There are two ways for representing binary tree, they are

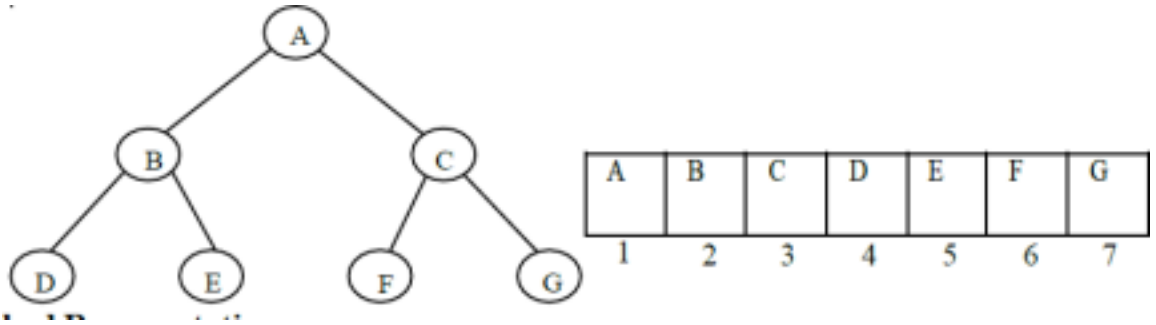
- Linear Representation
- Linked Representation

Linear Representation

• The elements are represented using arrays.

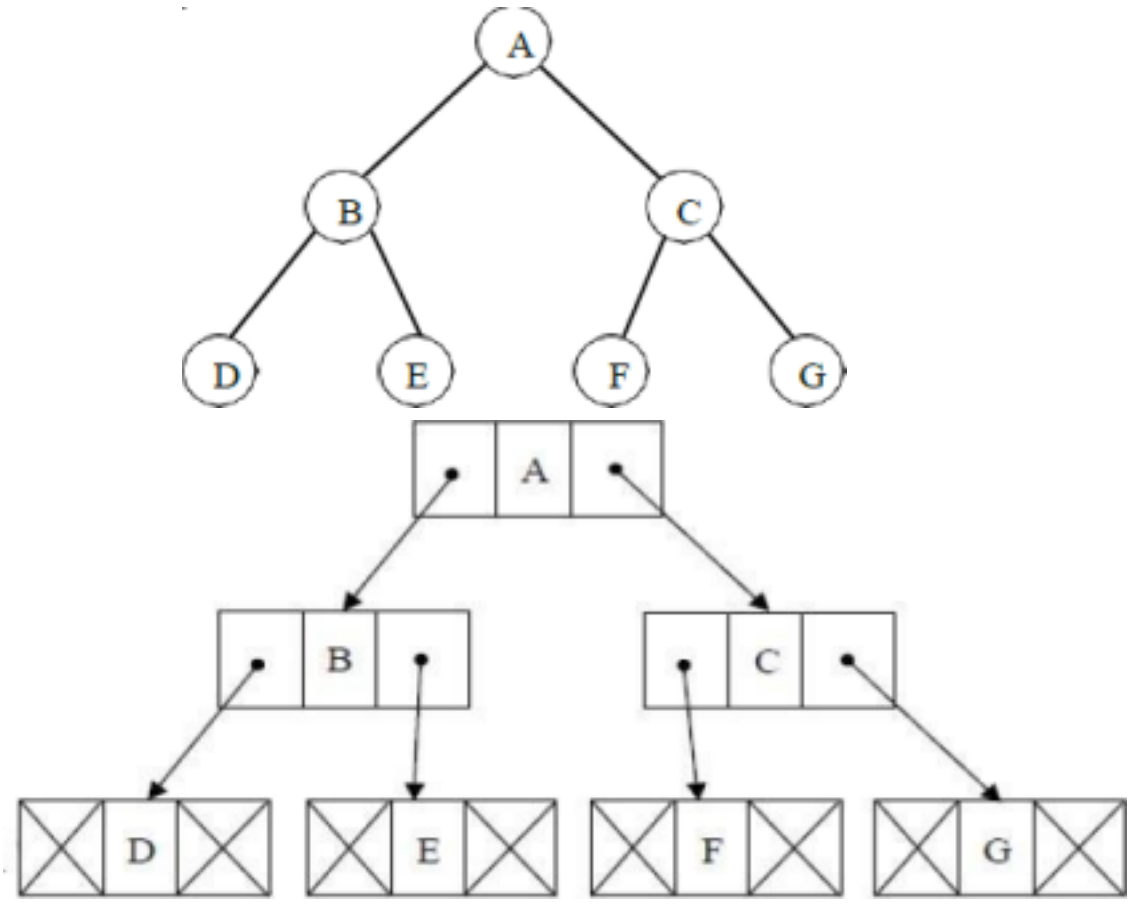
• For any element in position i, the left child is in position 2i, the right child is

in position (2i + 1), and the parent is in position (i/2).



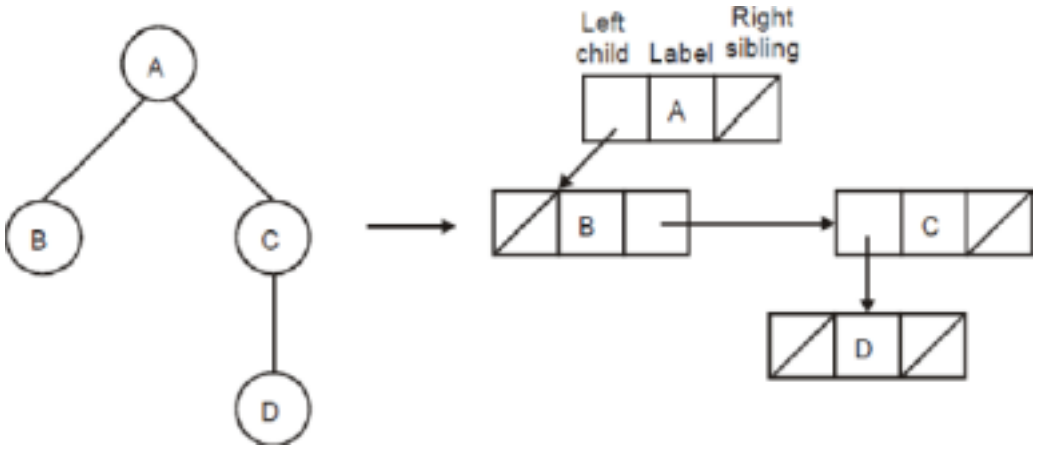
Linked Representation

- The elements are represented using pointers.
- Each node in linked representation has three fields, namely,
  - * Pointer to the left subtree
  - * Data field
  - * Pointer to the right subtree
- In leaf nodes, both the pointer fields are assigned as NULL.



The Leftmost-child, Right-sibling Data Structures

- In this representation, each node contains three fields namely, leftmost child, label and right sibling.
- Then, next pointers of node point to right siblings, and leftmost-child in cellspace.



2.4 EXPRESSION TREES

- Expression Tree is a binary tree in which the leaf nodes are operands and the interior nodes are operators.
- Like binary tree, expression tree can also be traversed by inorder, preorder and postorder traversal.

Constructing an Expression Tree

Let us consider postfix expression given as an input for constructing an expression tree

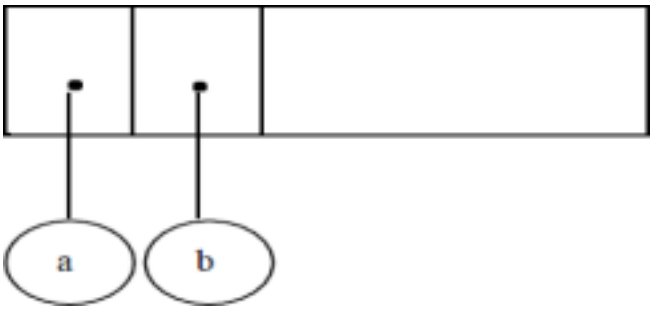
1. Read one symbol at a time from the postfix expression.
2. Check whether the symbol is an operand or operator.

(a) If the symbol is an operand, create a one - node tree and push a pointer on to the stack.

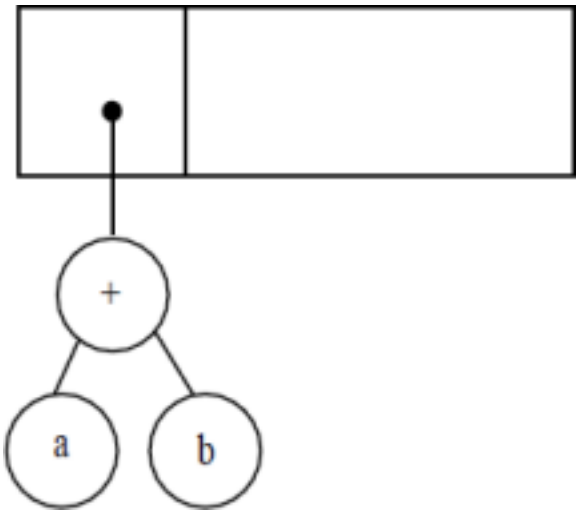
(b) If the symbol is an operator pop two pointers from the stack namely T1 and T2 and form a new tree with root as the operator and T2 as a left child and T1 as a right child. A pointer to this new tree is then pushed onto the stack. **Example: -**

ab + c *

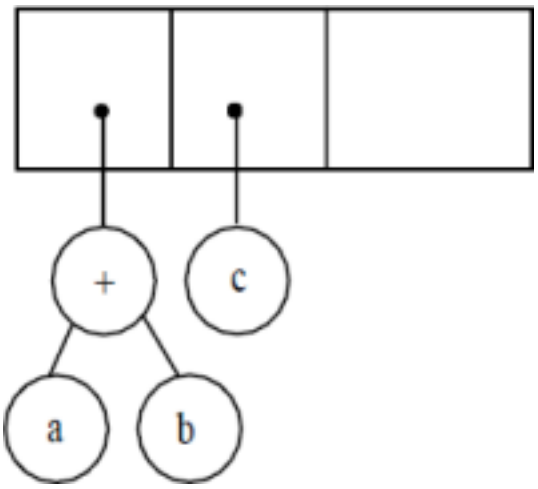
The first two symbols are operand, so create a one node tree and push the pointer on to the stack.



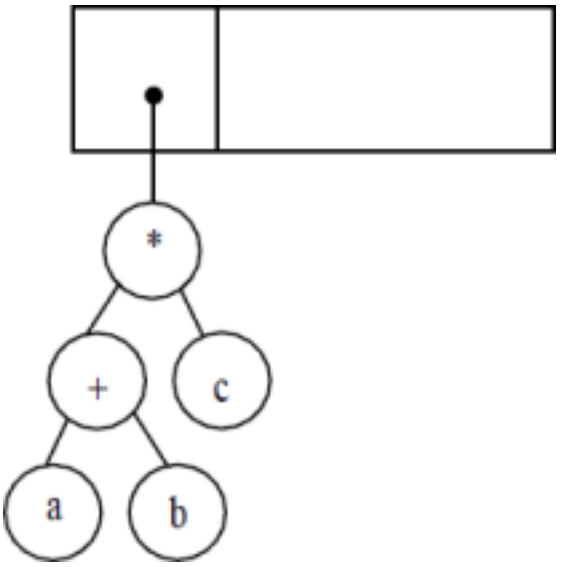
Next „+“ symbol is read, so two pointers are popped, a new tree is formed and a pointer to this is pushed on to the stack.



Next the operand C is read, so a one node tree is created and the pointer to it is pushed onto the stack.

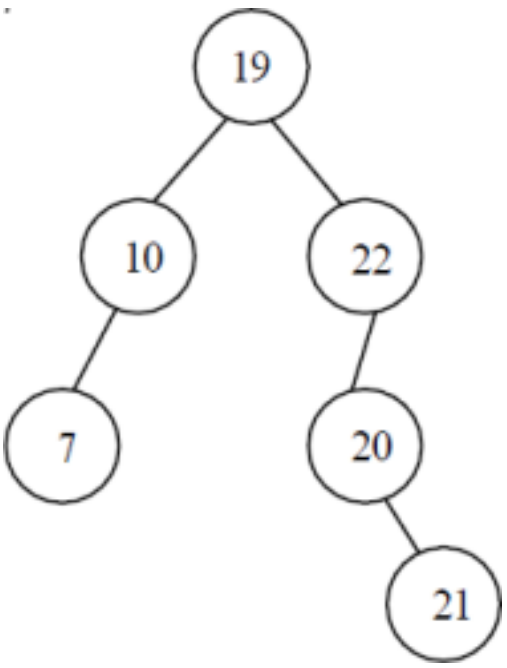


Now „*“ is read, so two trees are merged and the pointer to the final tree is pushed onto the stack



2.5 BINARY SEARCH TREE ADT

In a Binary search tree, the value of left node must be smaller than the parent node, and the value of right node must be greater than the parent node. This rule is applied recursively to the left and right subtrees of the root.



- Every BST is a Binary Tree
- But Not every binary tree is a BST

Declaration Routine for Binary Search Tree

```
class TreeNode:
 def __init__(self, key):
 self.key = key # The value stored in the node
 self.left = None # Reference to the left child node
 self.right = None # Reference to the right child node
```

Insert : -

To insert the element X into the tree,

* Check with the root node T

* If it is less than the root,

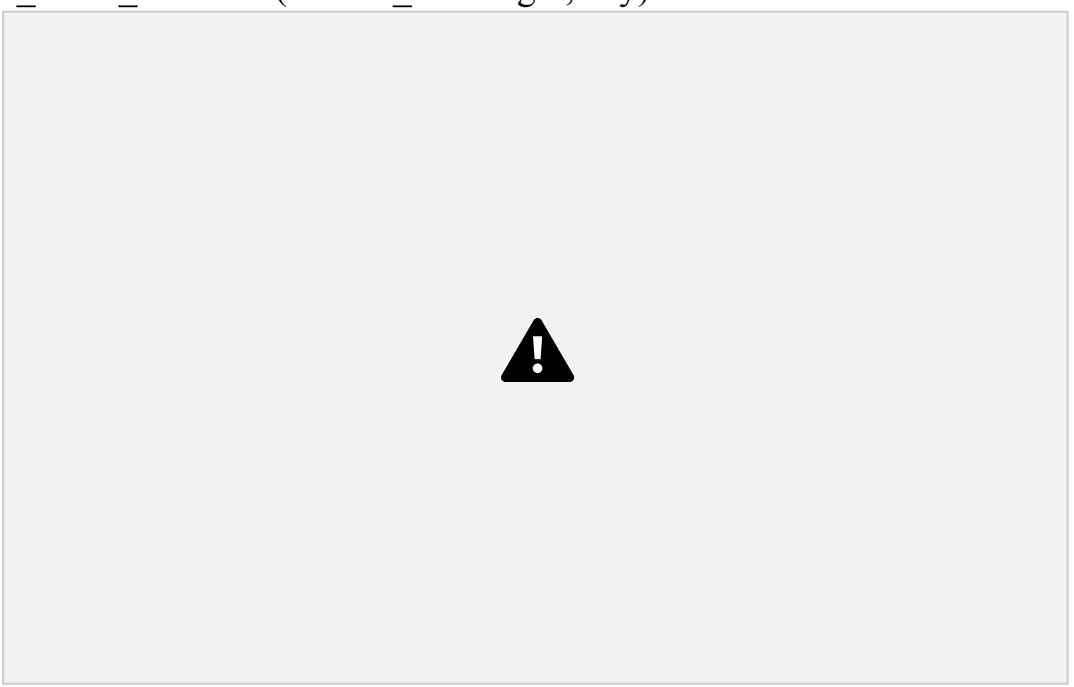
Traverse the left subtree recursively until it reachesthe T ->left equals to NULL. Then X is placed in T -> left.

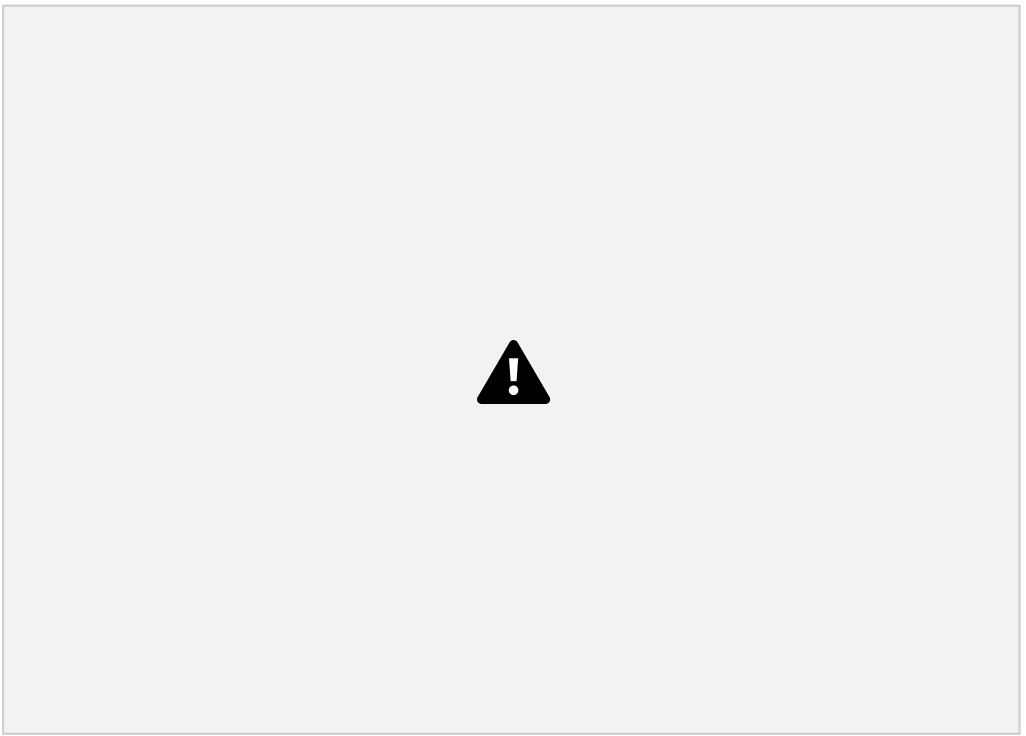
* If X is greater than the root.

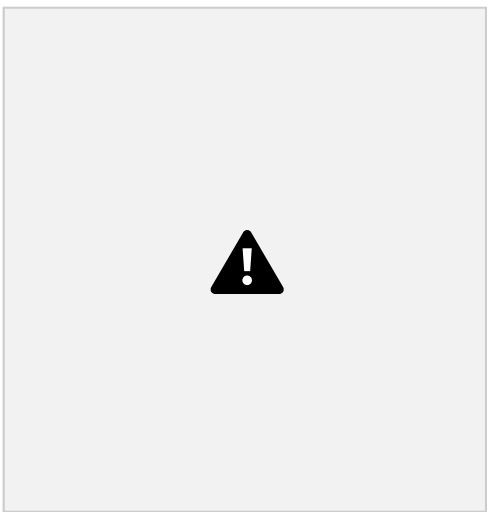
Traverse the right subtree recursively until it reaches the T -> right equals to NULL. Then X is placed in T-> Right.

Routine to Insert into a Binary Search Tree

```
def insert(self, key):
if self.root is None:
self.root = Node(key)
else:
self._insert_recursive(self.root, key)
def _insert_recursive(self, current_node, key):
if key < current_node.key:
if current_node.left is None:
current_node.left = Node(key)
else:
self._insert_recursive(current_node.left, key)
elif key > current_node.key:
if current_node.right is None:
current_node.right = Node(key)
else:
self._insert_recursive(current_node.right, key)
```







AFTER 3

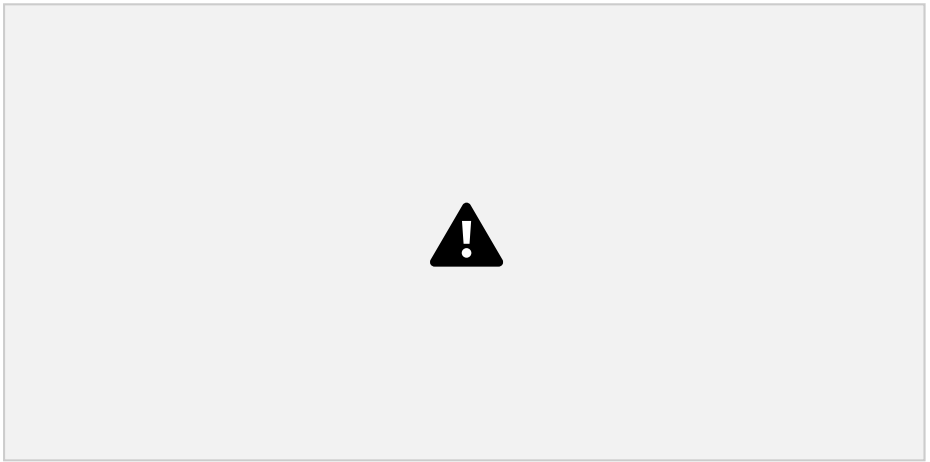
Find : -

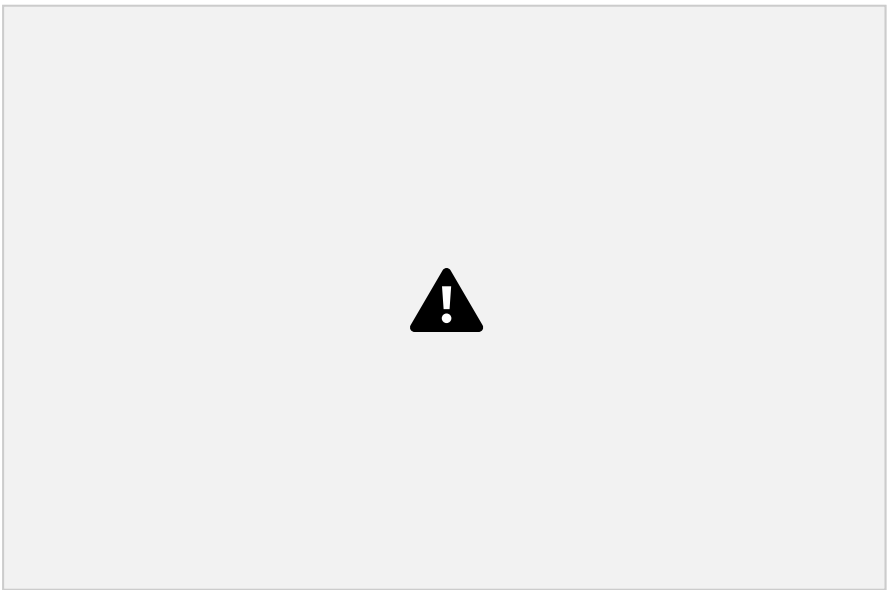
• Check whether the root is NULL if so then return NULL. • Otherwise, Check the value X with the root node value (i.e. T -> data) (1) If X is equal to T -> data, return T.

(2) If X is less than T -> data, Traverse the left of T recursively. (3) If X is greater than T -> data, traverse the right of T recursively.

Routine for find Operation

```
def search_recursive(root, key):
 # Base Case: root is None or key is present at root
 if root is None or root.key == key:
 return root
 # Key is greater than root's key, search in right subtree
 if root.key < key:
 return search_recursive(root.right, key)
 # Key is smaller than root's key, search in left subtree
 return search_recursive(root.left, key)
```



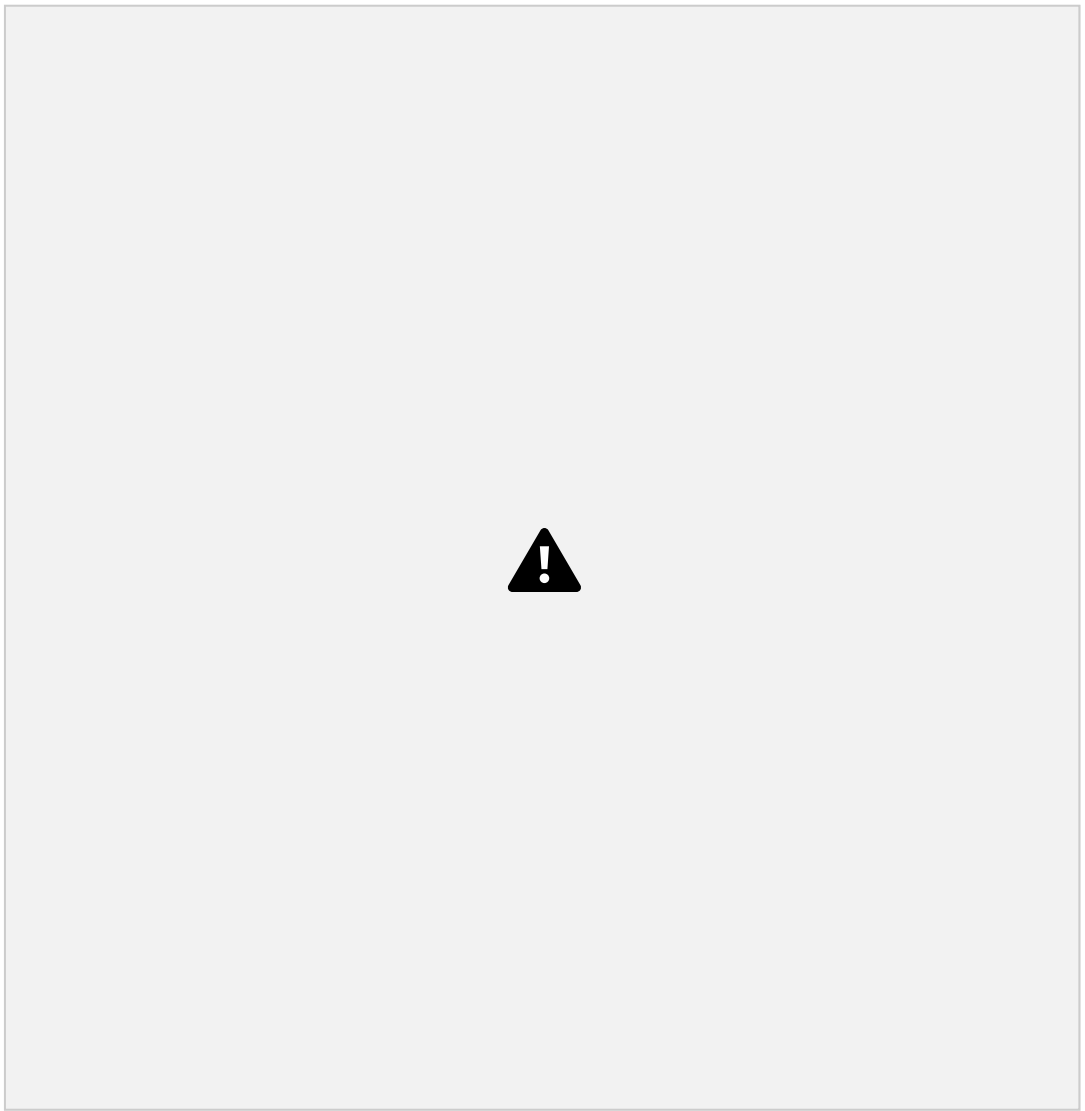


Find Min :

• This operation returns the position of the smallest element in the tree. • To perform FindMin, start at the root and go left as long as there is a left child. The stopping point is the smallest element.

Recursive routine to find minimum

```
def find_min_recursive(root):
 if root is None:
 return None # Handle empty tree
 # If there's no left child, the current node is the minimum
 if root.left is None:
 return root.data
 # Otherwise, the minimum is in the left subtree
 return find_min_recursive(root.left)
```

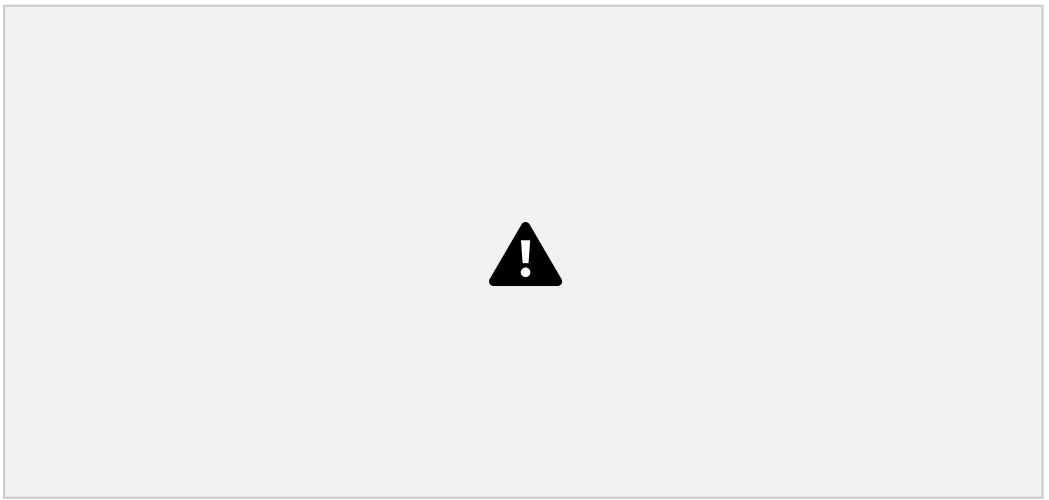


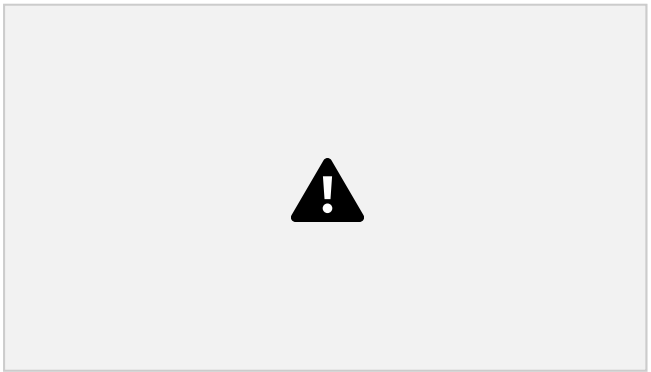
FindMax

FindMax routine return the position of largest elements in the tree. To perform a FindMax, start at the root and go right as long as there is a right child. The stopping point is the largest element.

Recursive routine to find maximum

```
def find_maximum_bst(root):
 if root is None:
 raise ValueError("The tree is empty.")
 current = root
 while current.right is not None:
 current = current.right
 return current.value
```





Delete :

Deletion operation is the complex operation in the Binary search tree. To delete an element, consider the following three possibilities.

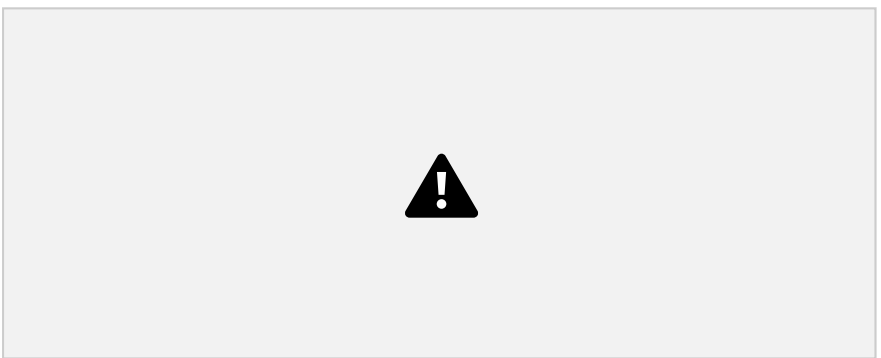
CASE 1: Node to be deleted is a leaf node (i.e) No children.

CASE 2: Node with one child.

CASE 3: Node with two children.

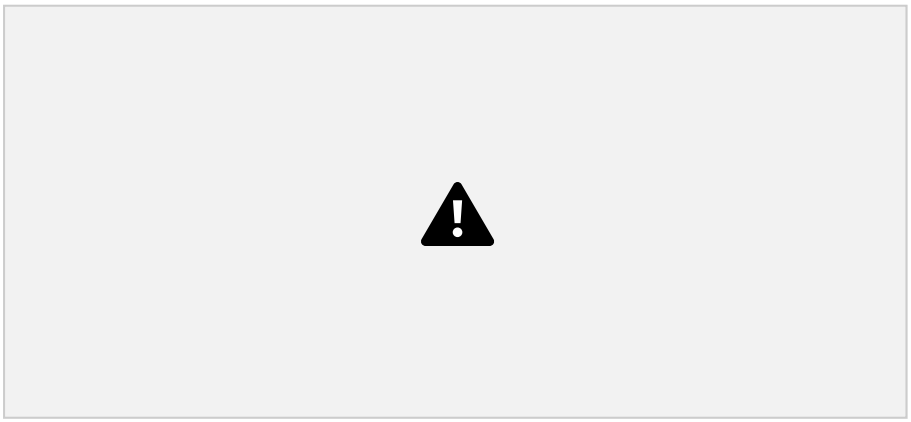
CASE 1 Node with no children (Leaf node)

If the node is a leaf node, it can be deleted immediately.



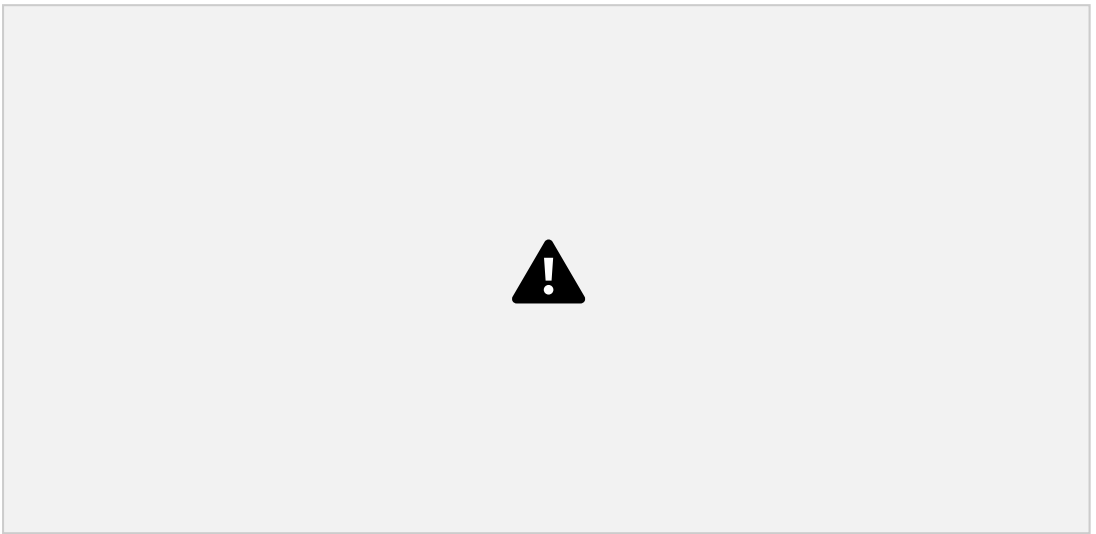
CASE 2 : - Node with one child

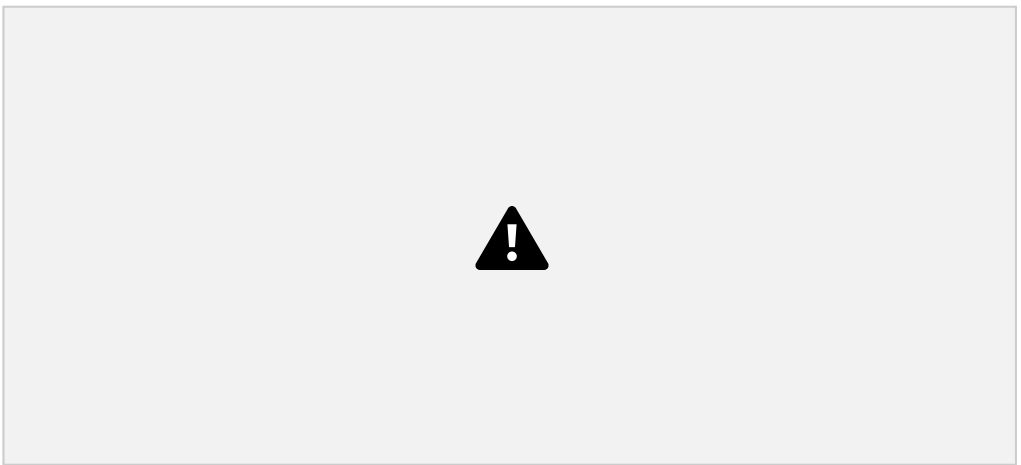
If the node has one child, it can be deleted by adjusting its parent pointer that points to its child node.

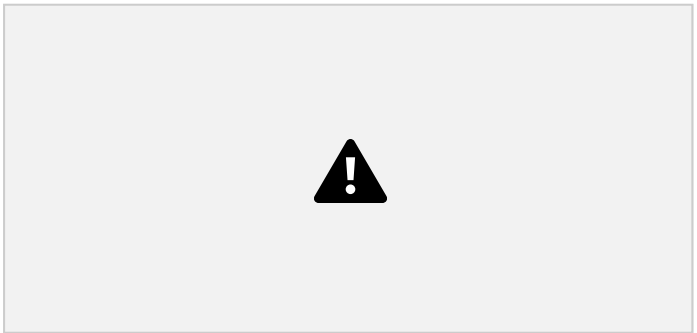


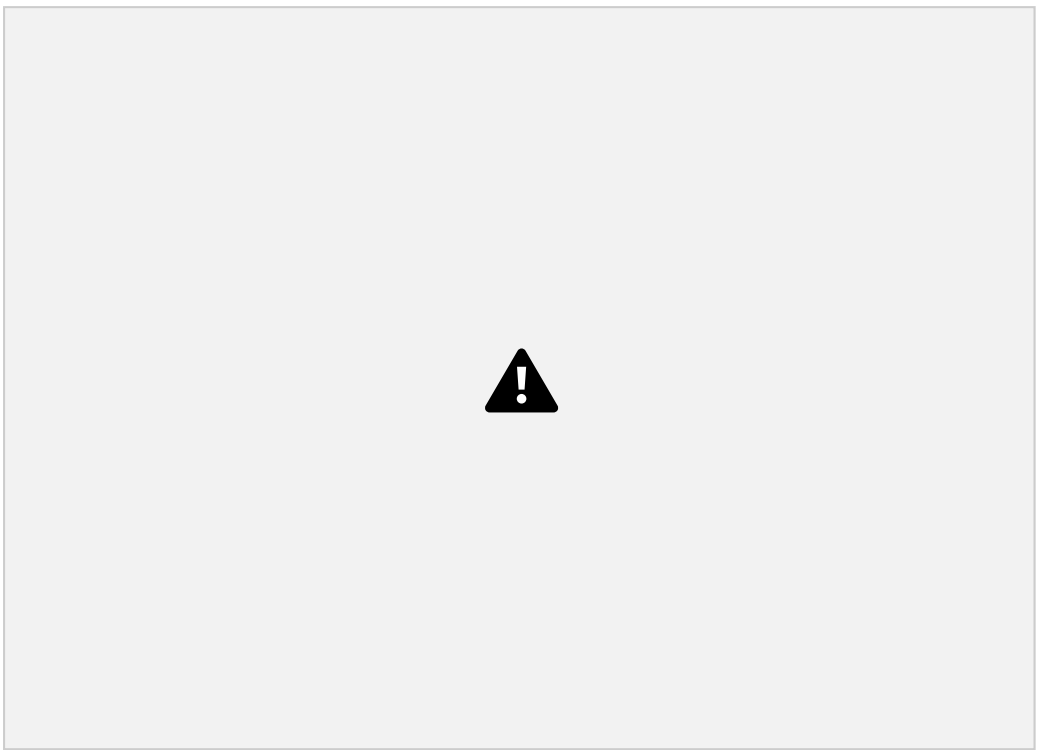
Case 3: Node with two children

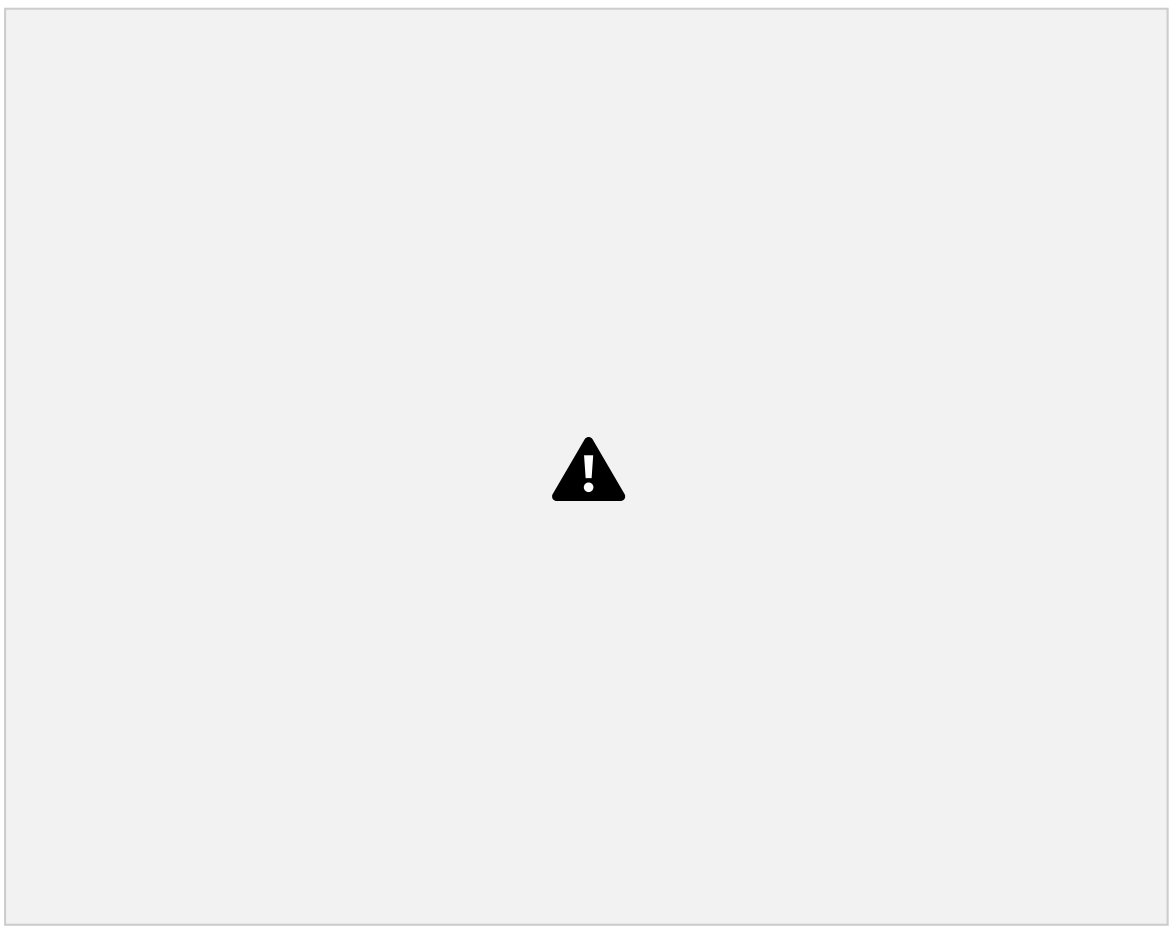
It is difficult to delete a node which has two children. The general strategy is to replace the data of the node to be deleted with its smallest data of the right subtree and recursively delete that node.

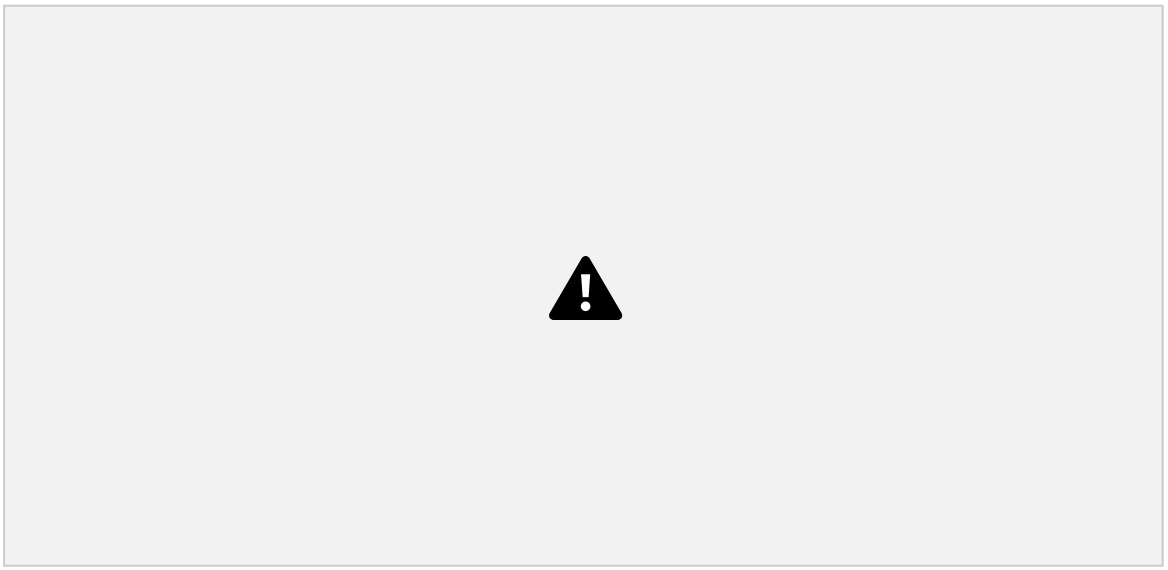












Routine for deletion from BST

SearchTree Delete (int X, searchTree T)

```
def delete_node(root, key):
 # Base Case: If the tree is empty or key not found
 if root is None:
 return root
# Traverse the tree to find the node to delete
 if key < root.key:
 root.left = delete_node(root.left, key)
 elif key > root.key:
 root.right = delete_node(root.right, key)
 else: # Node with the key is found
 # Case 1: Node with no children or one child
 if root.left is None:
 return root.right
 elif root.right is None:
 return root.left
 # Case 2: Node with two children
 # Find the in-order successor (smallest in the right subtree)
 temp = root.right
 while temp.left is not None:
 temp = temp.left
 # Copy the in-order successor's key to this node
 root.key = temp.key
 # Delete the in-order successor from the right subtree
```

root.right = delete_node(root.right, temp.key)

return root

2.6 AVL TREE

An AVL tree is a balanced binary search tree. In an AVL tree, balance factor of every node is either -1, 0 or +1. For every node in the tree, the heights of the two sub-trees may differ by at most one.

BALANCE FACTOR

The balance factor of a node is calculated by subtracting the height of its right sub tree from the height of its left sub-tree.

Every node has a balance factor of –1, 0, or 1.

BALANCE FACTOR = HEIGHT (LEFT SUB-TREE) –HEIGHT (RIGHT SUB TREE)

ROTATIONS

Rotation is the process of moving the nodes to either left or right to make tree balanced , If the balance factor of any node in an AVL tree becomes less than -1 or greater than 1

There are four rotations and they are classified into two types.

AN AVL TREE BECOMES UNBALANCED WHEN ONE OF THE FOLLOWING CONDITIONS OCCUR

1. An insertion into the left subtree of left child of node n
2. An insertion into the right subtree of right child of node n
3. An insertion into the right subtree of left child of node n
4. An insertion into the left subtree of right child of node n

In Cases 1 and 2 insertions occur at outside (Left-Left or Right-Right) and the unbalance condition is balanced by Single Rotation

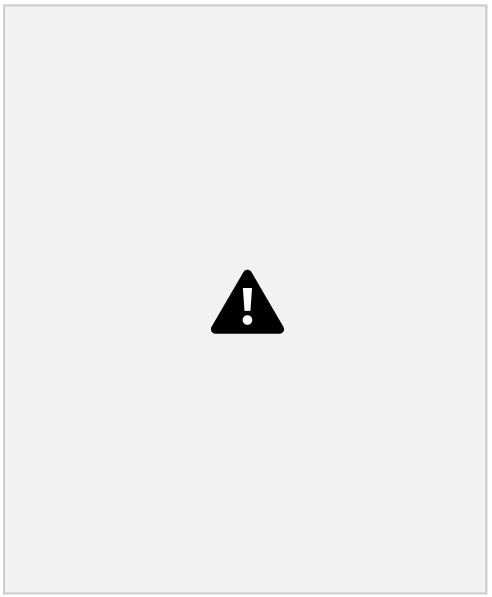
In Cases 3 and 4 insertions occur at inside (Left-Right or Right-Left) and the unbalance condition is balanced by Double Rotation

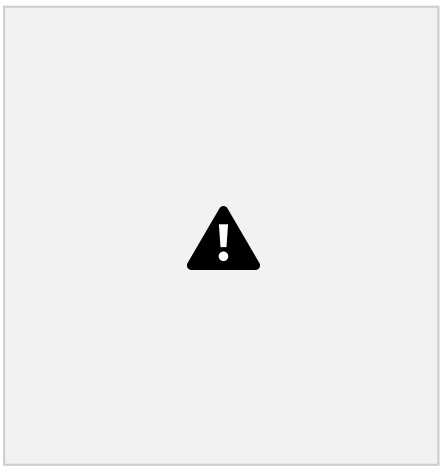
SINGLE ROTATION

Single rotation occurs for case 1&2 i.e

• An insertion into the left subtree of left child of node and (LL Rotation) • An insertion into the right subtree of right child of node n(RR Rotation) Case 1. An insertion into the left subtree of the left child of K2. Single Rotation to fix Case 1.

General Representation



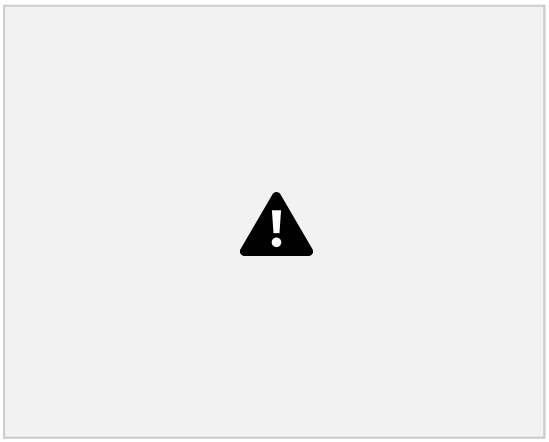


Routine to Perform Single Rotation with Left

```
def left_rotate(z):
 y = z.right # y becomes the new root
 T2 = y.left # T2 is y's left child, which will become z's right child
# Perform the rotation
 y.left = z
 z.right = T2
 # Update heights of affected nodes
 update_height(z)
 update_height(y)
 return y # Return the new root of the subtree
```

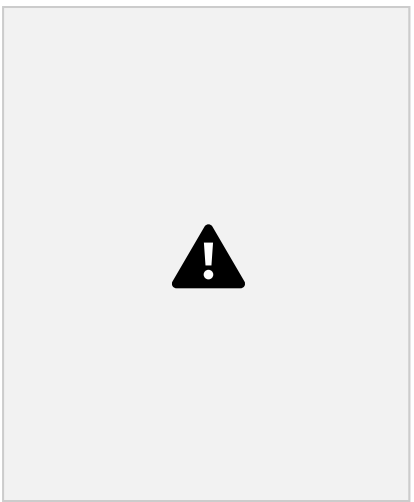
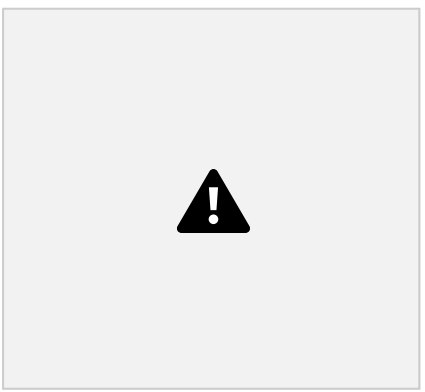
Example :

Inserting the value ‘1’ in the following AVL Tree makes AVL Tree imbalance




Single Rotation to fix Case 4 :-

Case 4 : - An insertion into the right subtree of the right child of K1 **General Representation**

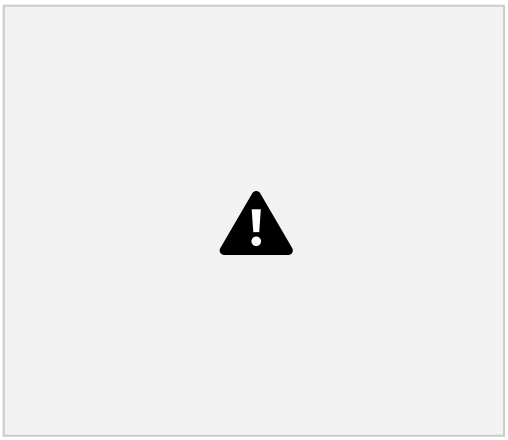
Routine to Perform Single Rotation with Right :

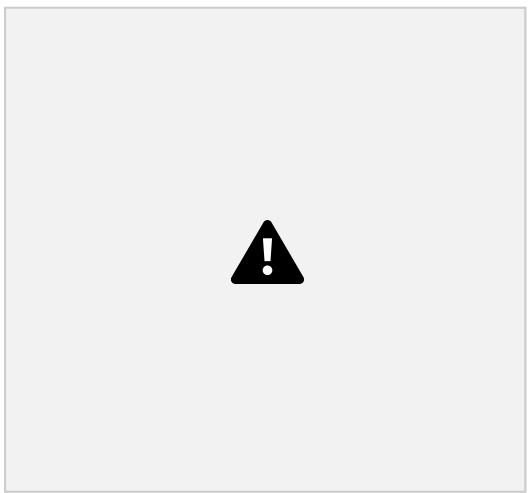
```
def right_rotate(z):
 y = z.left
 T2 = y.right
 # Perform rotation
 y.right = z
 z.left = T2
 # Update heights
 update_height(z)
 update_height(y)
 # Return the new root of the subtree
```

return y

Example : -

Inserting the value ‘10’ in the following AVL

Tree. 

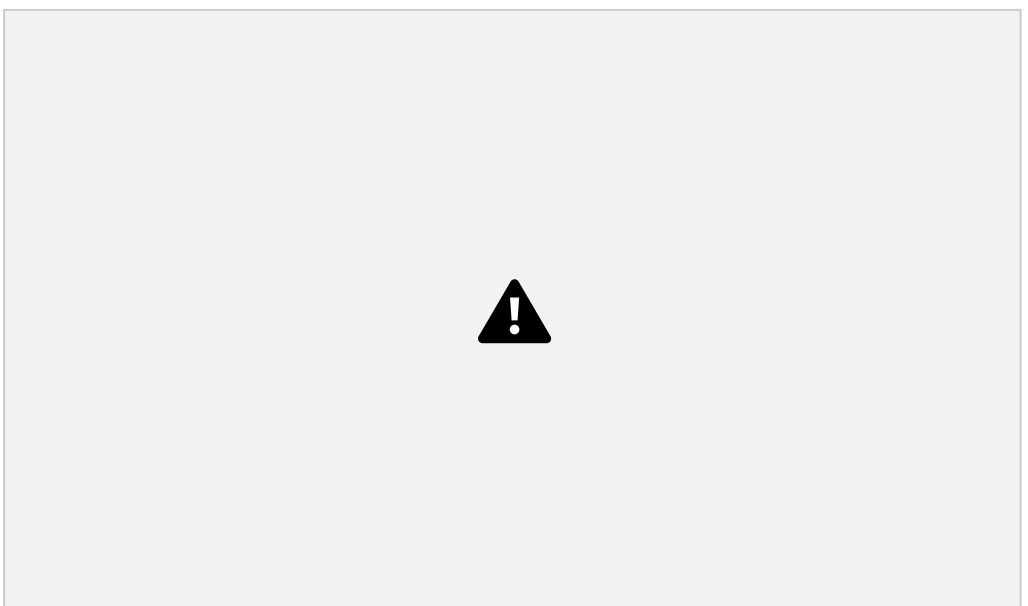


Double Rotation

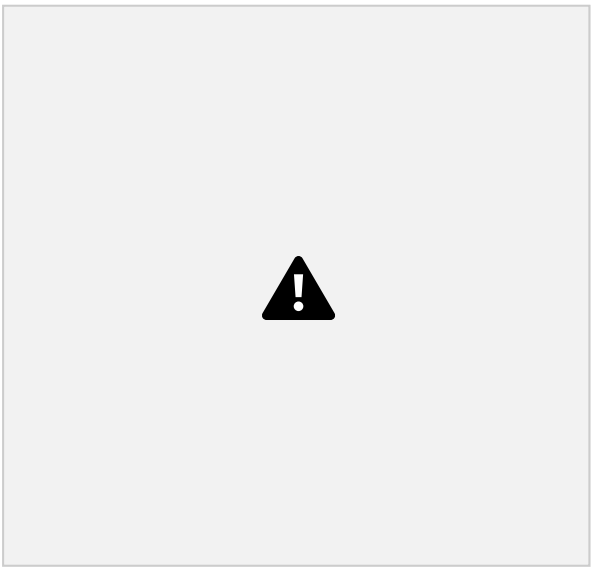
Double Rotation is performed to fix case 2 and case 3.

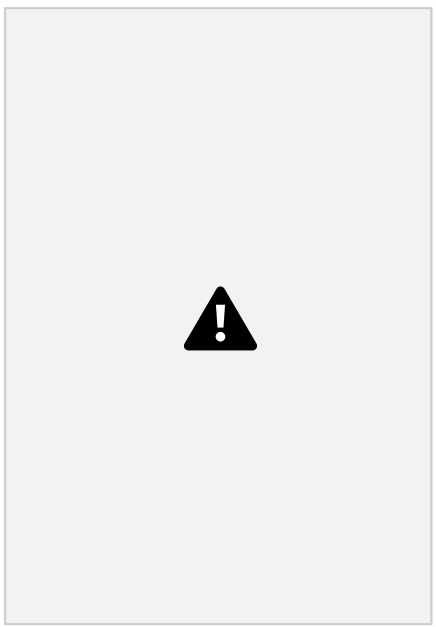
Case 2 : An insertion into the right subtree of the left child.

General Representation



This can be performed by 2 single rotations.





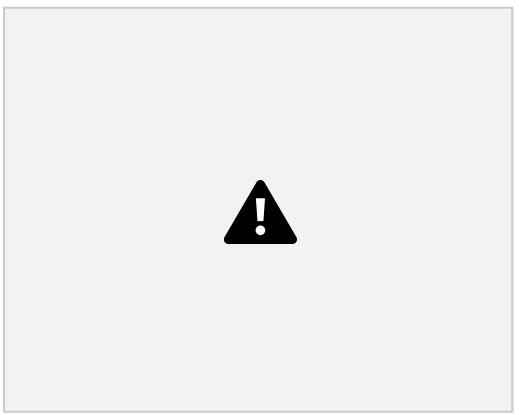


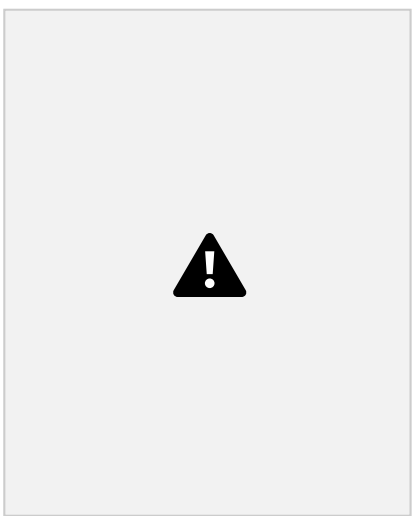
Fig. 3.8.7 Balanced AVL Tree

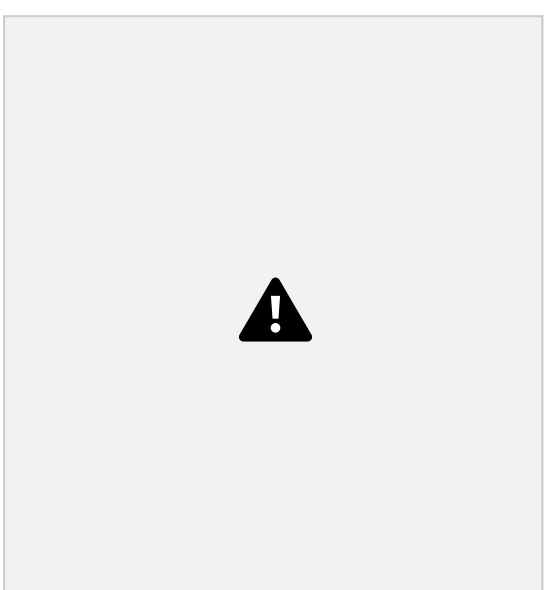
Routine to Perform Double Rotation with Left :

```
def rotate_left(z):
 y = z.right
 T2 = y.left
 y.left = z
 z.right = T2
 # Update heights
 z.height = 1 + max(getHeight(z.left), getHeight(z.right))
```

y.height = 1 + max(getHeight(y.left), getHeight(y.right))
return y

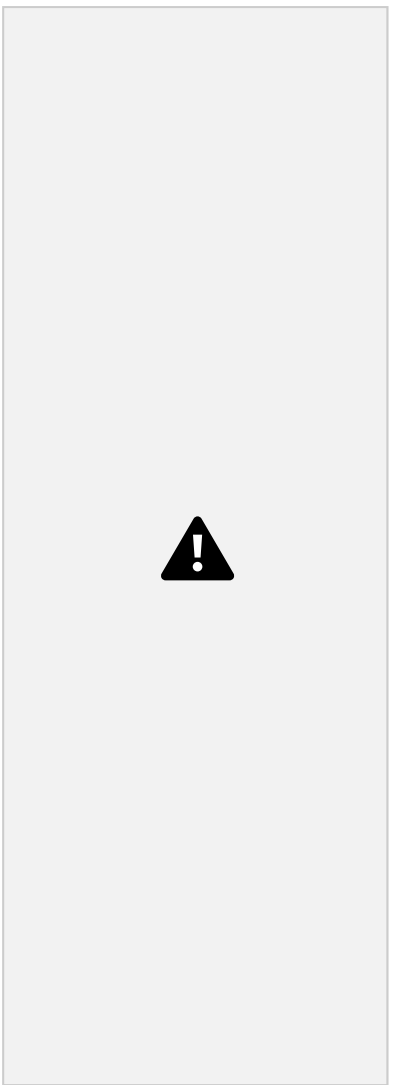
Example: Insertion of either ‘12’ or ‘18’ makes AVL Tree

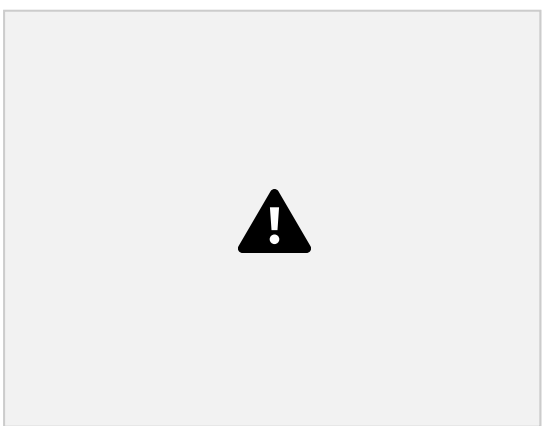
imbalance 



(b) After rotation

This can be done by performing single rotation with right of ‘10’ & then perform the single rotation with left of 20 as shown below.

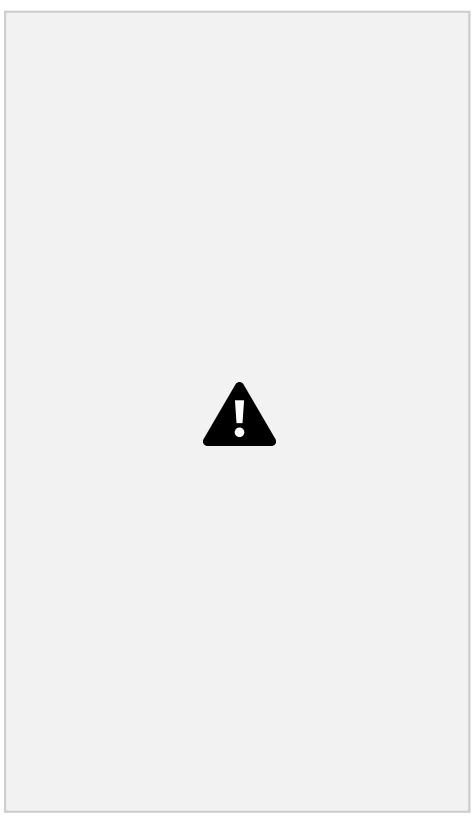


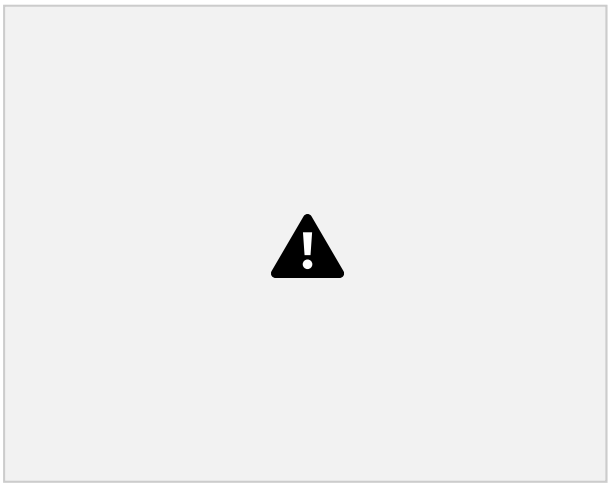


Case 4 : An Insertion into the left subtree of the right child of K1.

General Representation: -

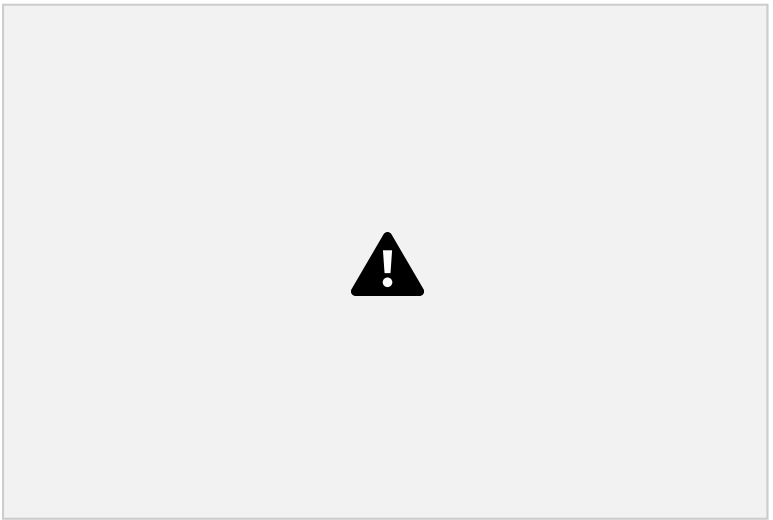
This can also be done by performing single rotation with left and then single rotation with right.

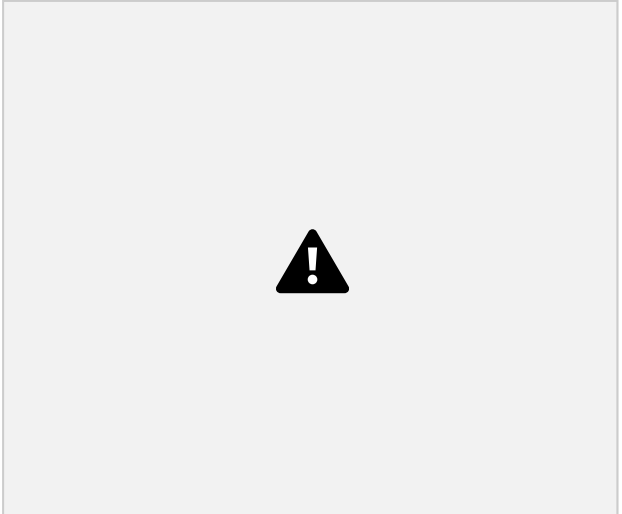




Routine to Perform Double Rotation with Right:

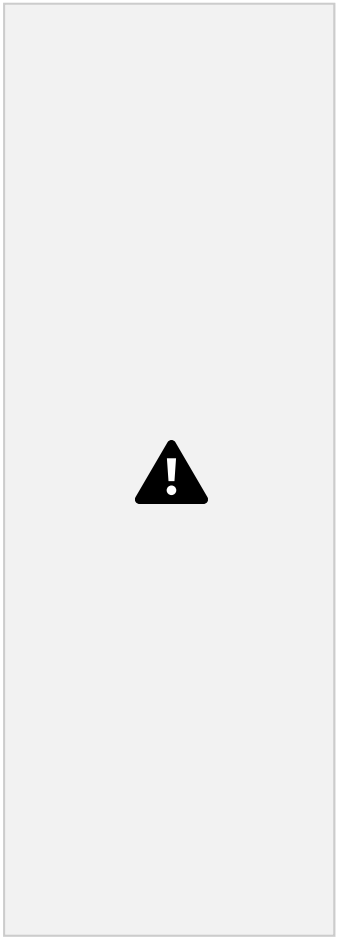
```
def rotate_right(z):
 y = z.left
 T3 = y.right
 y.right = z
 z.left = T3
 # Update heights
 z.height = 1 + max(getHeight(z.left), getHeight(z.right))
y.height = 1 + max(getHeight(y.left), getHeight(y.right))
return y
```

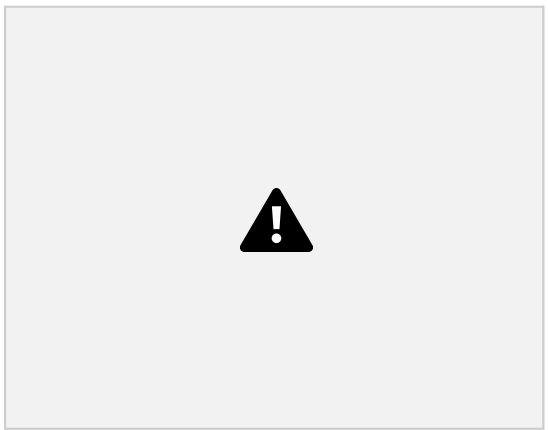




(b) After

This can be done by performing single rotation with left of ‘15’ and then performing the single rotation with right of ‘10’ as shown below





Routine to Insert in an AVL Tree :

```
class AVLNode:
 def __init__(self, key):
 self.key = key
 self.left = None
 self.right = None
 self.height = 1

def insert(self, root, key):
 # 1. Standard BST Insertion  if
```

```
not root:
 return AVLNode(key)
 elif key < root.key:
 root.left = self.insert(root.left, key)
 else:
 root.right = self.insert(root.right, key)
 # 2. Update Height
 root.height = 1 + max(self.getHeight(root.left), self.getHeight(root.right))  #
3. Get Balance Factor
 balance = self.getBalance(root)
 # 4. Perform Rotations if needed
 # Left Imbalance
 if balance > 1:
 if key < root.left.key: # Left-Left Case
 return self.rightRotate(root)
 else: # Left-Right Case
 root.left = self.leftRotate(root.left)
 return self.rightRotate(root)
 # Right Imbalance
 elif balance < -1:
 if key > root.right.key: # Right-Right Case
 return self.leftRotate(root)
 else: # Right-Left Case
 root.right = self.rightRotate(root.right)
 return self.leftRotate(root)
 return root
```

2.7 TREE TRAVERSALS

- Traversing means visiting each node only once.
- Tree traversal is a method for visiting all the nodes in the tree exactly once.

There are three types of tree traversal techniques

a) Inorder Traversal

b) Preorder Traversal

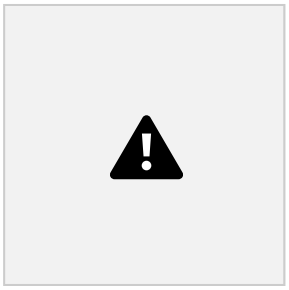
c) Postorder Traversal

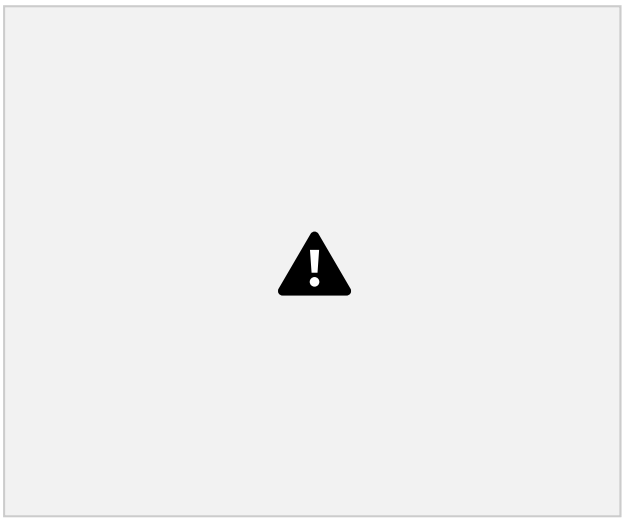
a) Inorder Traversal

The inorder traversal of a binary tree is performed as

- Traverse the left subtree in inorder
- Visit the root
- Traverse the right subtree in inorder.

Example :





Recursive routine for inorder traversal

```
def inorder_traversal_recursive (root):
result = []

 def traverse (node):
 if node is None:
 return
 # 1. Traverse the left subtree
 traverse (node.left)
 # 2. Visit the current node
 result.append (node.val)
 # 3. Traverse the right subtree
```

```
 traverse (node.right)
 traverse (root)
 return result
```

b) Preorder Traversal

The preorder traversal of a binary tree is performed as follows, • Visit the root

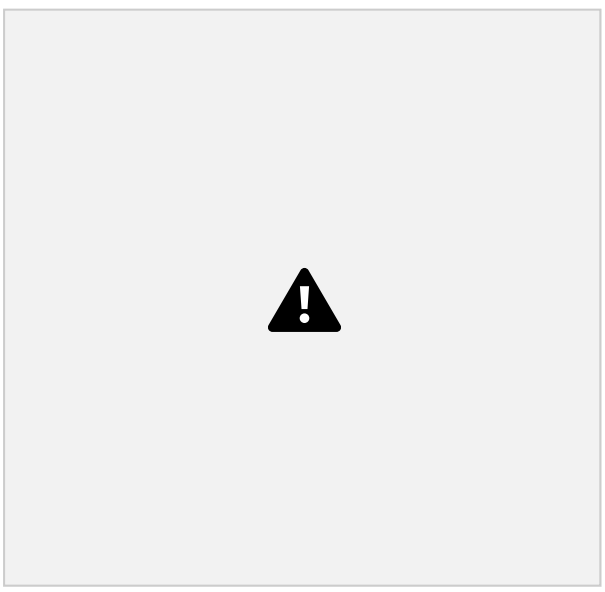
• Traverse the left subtree in preorder

• Traverse the right subtree in preorder.

Example:

Recursive Routine For Preorder Traversal

```
def preorder_traversal_recursive(root,
result=None):  if result is None:
 result = []
```



```
 if root:
 # 1. Visit the Root Node
 result.append(root.value) # Or print(root.value)  #
2. Traverse the Left Subtree
 preorder_traversal_recursive(root.left, result)
```

```
 # 3. Traverse the Right Subtree
 preorder_traversal_recursive(root.right, result)
 return result
```

c)Postorder Traversal

The postorder traversal of a binary tree is performed by the following steps. • Traverse the left subtree in postorder.

• Traverse the right subtree in postorder.

• Visit the root.

Example : 1

Postorder : - 10, 30, 20

Example : 2

Recursive Routine For Postorder Traversal

```
def postorder_traversal(node, result):
 # Base case: if the node is None, return (end of a branch)  if
node is None:
 return
 # Recursively traverse the left subtree
 postorder_traversal(node.left, result)
 # Recursively traverse the right subtree
 postorder_traversal(node.right, result)
 # Process the current node (add its data to the result list)
 result.append(node.data)
```

2.8 PRIORITY QUEUE (HEAPS) – BINARY HEAP

- In a priority queue, an element with high priority is served before an element with low priority.
- If two elements have the same priority, they are served according to their order in the queue.

Two types of priority queue

1. Max Priority Queue
2. Min Priority Queue

Max Priority Queue

In Max Priority Queue, elements are inserted in the order in which they arrive they queue and always maximum value is removed first from the queue. E.x : insert in order 8, 3, 2, 5 removed in the order 8, 5, 3, 2

Min Priority Queue

Min Priority Queue is similar to Max priority queue except removing maximum elements first, we remove min. element first in min priority queue

BINARY HEAP

- The efficient way of implementing priority queue is Binary Heap.
- Binary heap is merely referred as Heaps

Heap have two properties namely

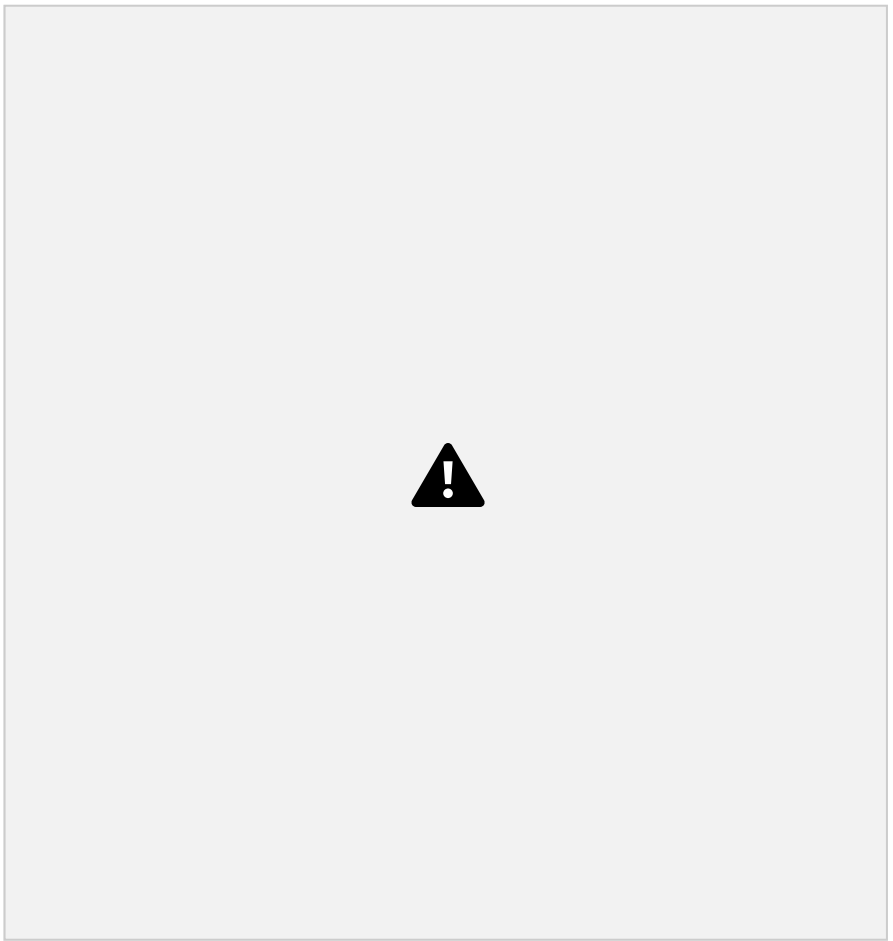
- Structure property
- Heap order property.

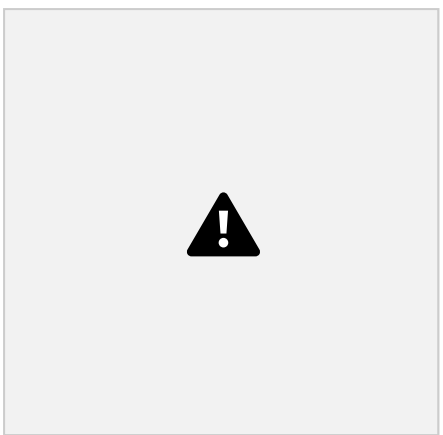
Structure Property

- A heap should be complete binary tree, which is a completely filled binary tree with the possible exception of the bottom level, which is filled from left to right.
- A complete binary tree of height H has between $2^H$and 2^{H+1}-1 nodes. • For example, if the height is 3. Then the number of nodes will be between 8 and 15. (ie) ($2^3$and 2^4-1).
- For any element in array position i, the left child is in position 2i, the right

child is in position 2i + 1, and the parent is in i/2.

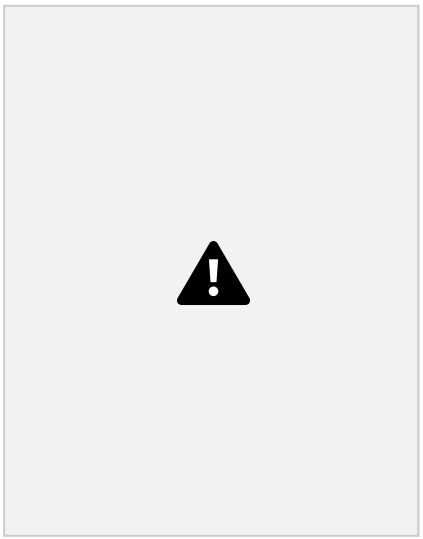
• As it is represented as array it doesn’t require pointers and also the operations required to traverse the tree are extremely simple and fast. • But the only disadvantage is to specify the maximum heap size in advance.

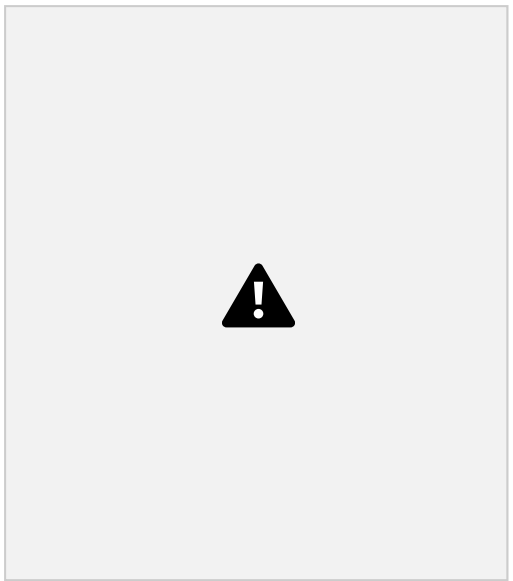




Heap Order Property

- In a heap, for every node X, the key in the parent of X is smaller than (or equal to) the key in X.
- This property allows the deletemin operations to be performed quickly has the minimum element can always be found at the root.
- Thus, we get the FindMin operation in constant time.



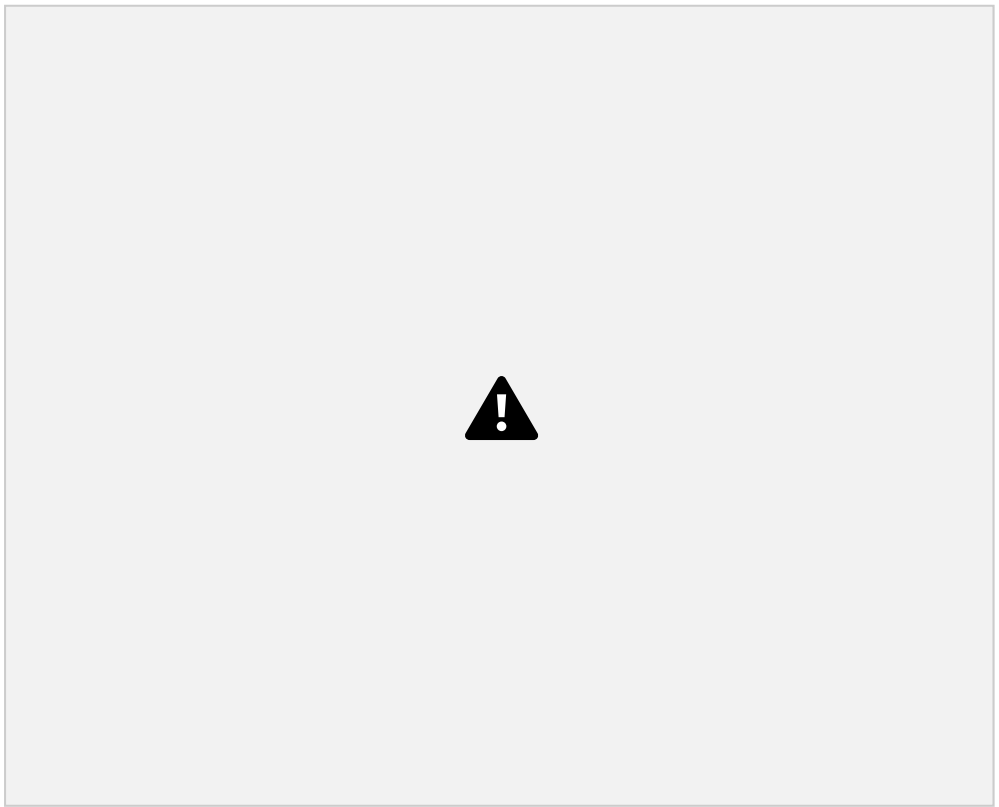


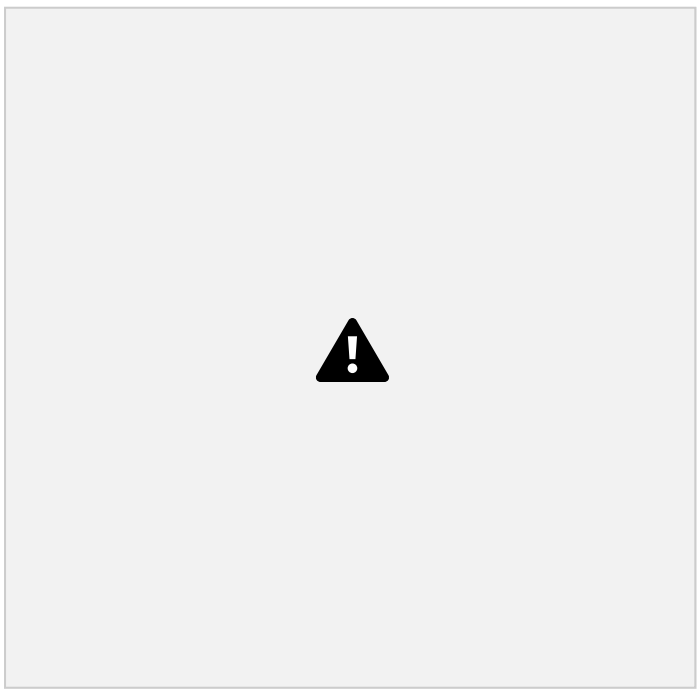
Basic Heap Operations

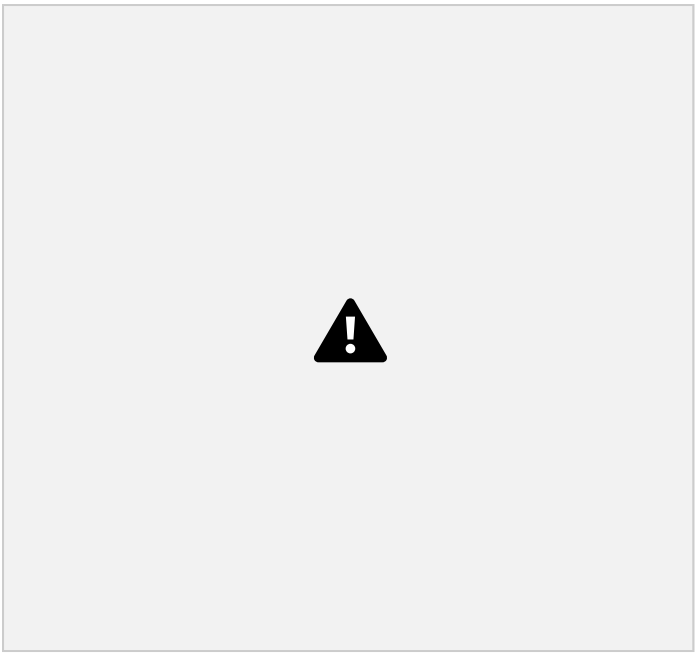
To perform the insert and DeleteMin operations ensure that the heap order property is maintained.

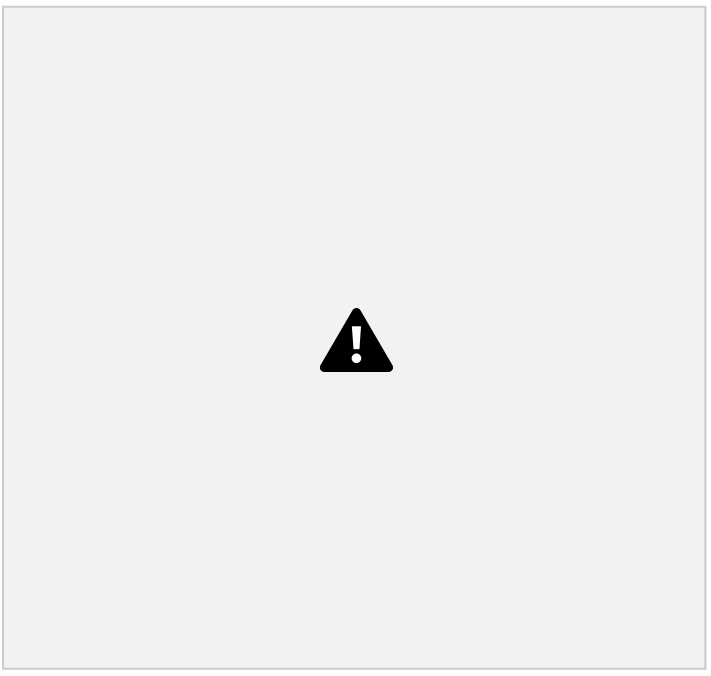
Insert Operation

- To insert an element X into the heap, we create a hole in the next available location, otherwise the tree will not be complete.
- If X can be placed in the hole without violating heap order, then place the element X there itself.
- Otherwise, we slide the element that is in the hole's parent node into the hole, thus bubbling the hole up toward the root.
- This process continues until X can be placed in the hole.
- This general strategy is known as Percolate up, in which the new element is percolated up the heap until the correct location is found.









In Fig (d) the value 10 is placed in its correct location.

DeleteMin

• DeleteMin Operation is deleting the minimum element from the Heap.

• In Binary heap the minimum element is found in the root.

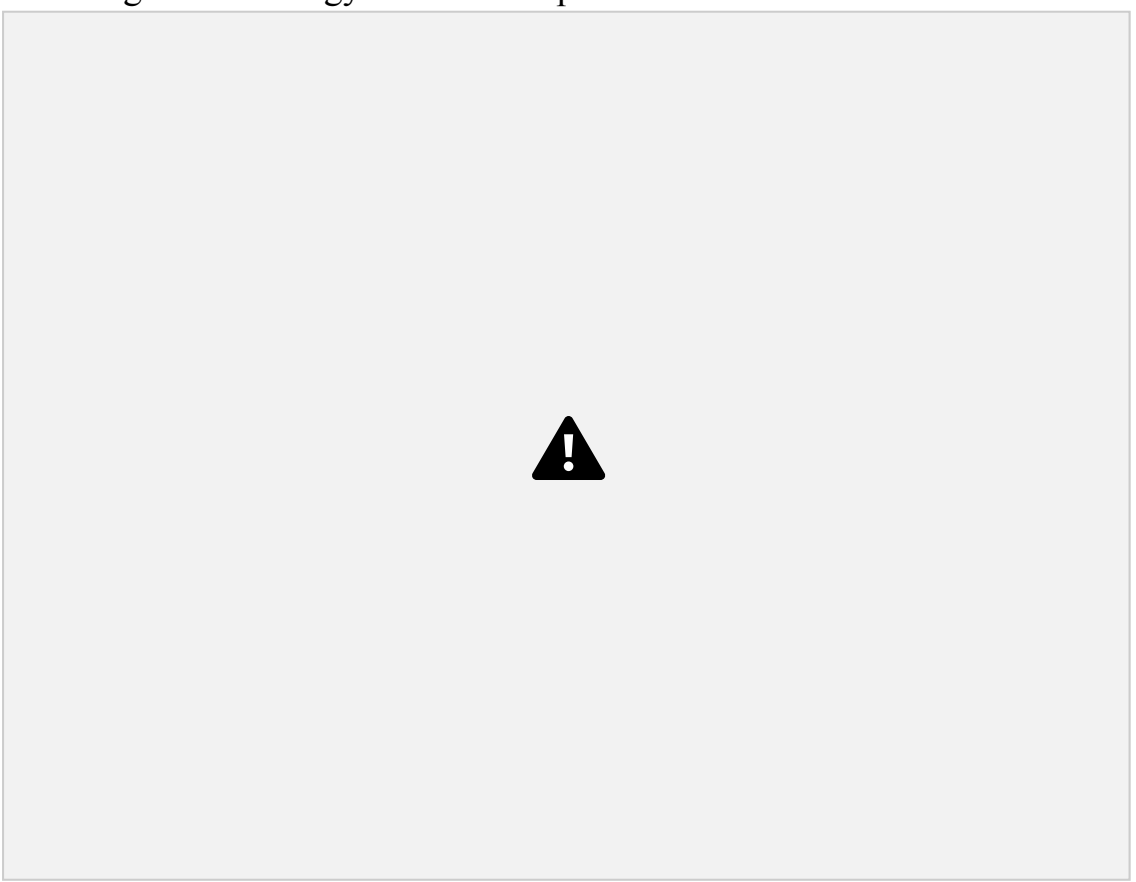
• When this minimum is removed, a hole is created at the root. • Since the heap becomes one smaller, makes the last element X in the heap to move somewhere in the heap.

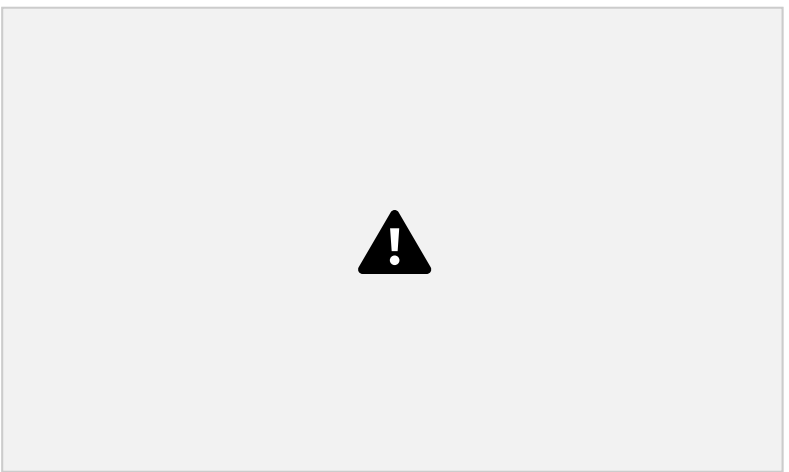
• If X can be placed in hole without violating heaporder property place it. • Otherwise, we slide the smaller of the hole's children into the hole, thus

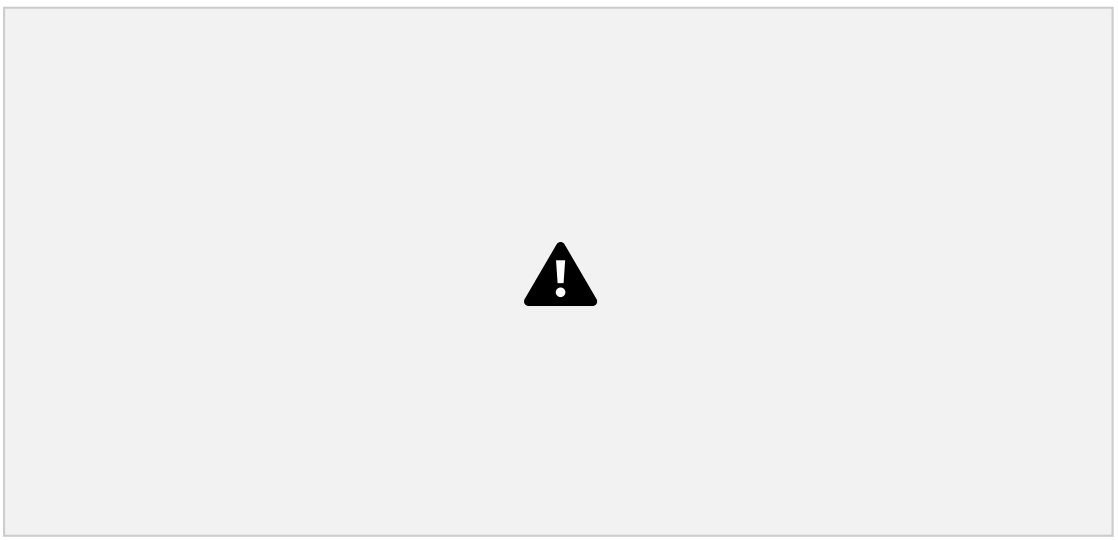
pushing the hole down one level.

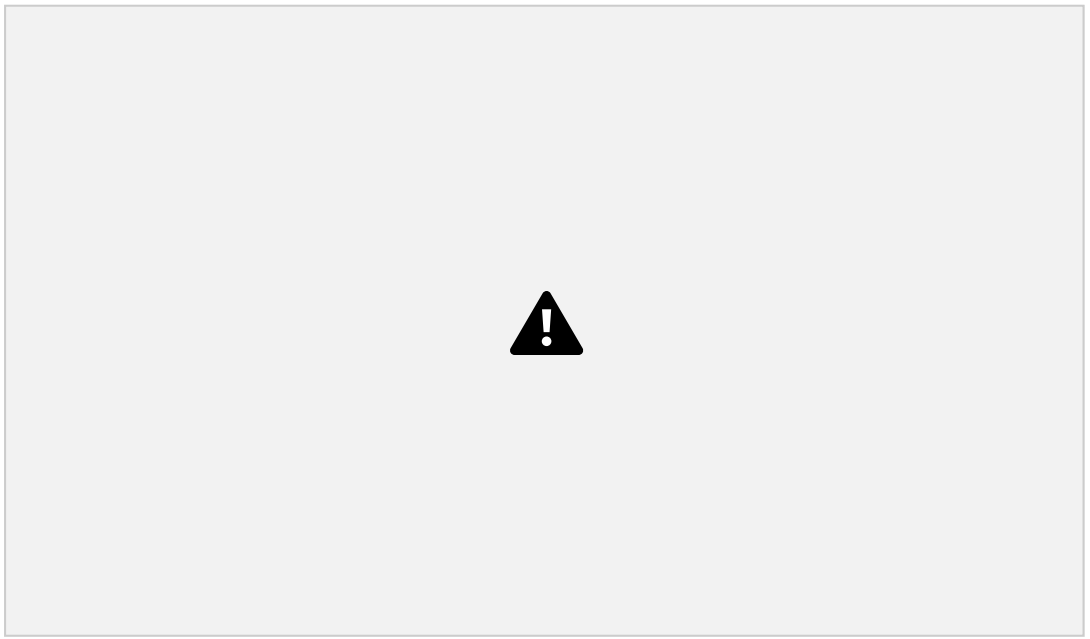
• We repeat until X can be placed in the hole.

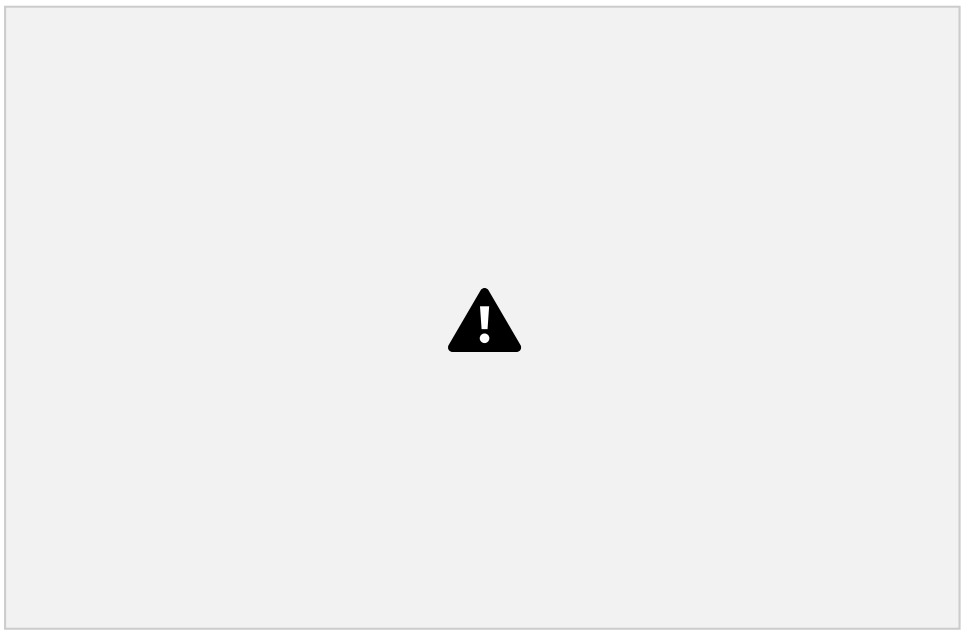
• This general strategy is known as percolate down.











```
# Priority Queue implementation in Python

# Function to heapify the tree

def heapify(arr, n, i):

 # Find the largest among root, left child, and right child

largest = i

 l = 2 * i + 1

 r = 2 * i + 2

 if l < n and arr[i] < arr[l]:

 largest = l

 if r < n and arr[largest] < arr[r]:

 largest = r

 # Swap and continue heapifying if root is not the largest
```

```
if largest != i:
 arr[i], arr[largest] = arr[largest], arr[i]
 heapify(arr, n, largest)
# Function to insert an element into the tree
def insert(array, newNum):
 size = len(array)
 if size == 0:
 array.append(newNum)
 else:
 array.append(newNum)
 for i in range((size // 2) - 1, -1, -1):
 heapify(array, size, i)
# Function to delete an element from the
tree def deleteNode(array, num):
 size = len(array)
 i = 0
 for i in range(0, size):
 if num == array[i]:
 break
 # Swap the element to delete with the last element
 array[i], array[size - 1] = array[size - 1], array[i]
 # Remove the last element (the one we want to delete)
 array.pop()
 # Rebuild the heap
 for i in range((len(array) // 2) - 1, -1, -1):
 heapify(array, len(array), i)
arr = []
insert(arr, 3)
insert(arr, 4)
insert(arr, 9)
```

```
insert(arr, 5)

insert(arr, 2)

print("Max-Heap array: " + str(arr))

deleteNode(arr, 4)

print("After deleting an element: " + str(arr))
```

APPLICATIONS OF HEAP

- To quickly find the smallest and largest element from a collection of items or array.
- In the implementation of Priority queue in graph algorithms like Dijkstra's algorithm (shortest path), Prim's algorithm (minimum spanning tree) and Huffman encoding (data compression).
- In order to overcome the Worst Case Complexity of Quick Sort algorithm from O(n^2) to O(nlog(n)) in Heap Sort.
- For finding the order in statistics.
- Systems concerned with security and embedded system such as Linux Kernel uses Heap Sort because of the O(nlog(n)) .

2.9 TRIE (PREFIX TREE) AND REAL-TIME USE IN AUTOCOMPLETE AND SPELL CHECK

- A Trie, also known as a Prefix Tree, is a tree-like data structure used for efficient storage and retrieval of strings. It is particularly useful for operations involving prefixes, such as autocomplete and spell checkers.
- Derive from "retrieval."

Representation of Trie Node

- Trie data structure consists of nodes connected by edges.
- Each node represents a character or a part of a string.
- The root node acts as a starting point and does not store any character.

Node declaration in Trie

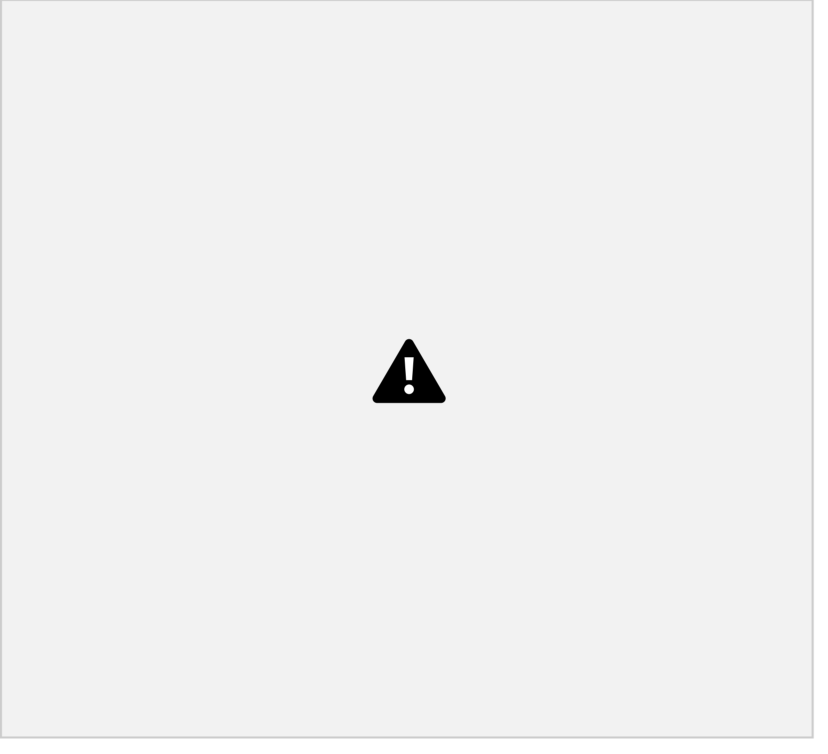
```
class TrieNode:
        def __init__(self):
                self.children = [None] * 26
                self.isEndOfWord = False
```

Structure

Nodes: Each node in a Trie represents a single character.

Root Node: The root node typically represents an empty string.

Children: Each node contains a collection (often a dictionary or array) of child nodes, where each child corresponds to the next character in a potential word. **End of Word Flag:** Each node also contains a boolean flag indicating whether the path leading to that node represents a complete word.



Tries Example

S = { bear, bell, bid, bull, buy, sell, stock, stop }

Example: standard trie for the set of strings

e b i e t

u
y
s

a l l

l
o

r d
l

c
k
p

l l

Operations:

insert(word):

Traverses the Trie, creating new nodes for characters not yet present and marking the final node of the word as

is_end_of_word = True.

search(word):

Traverses the Trie based on the input word. Returns True if the entire word is found and its last character's node is marked as an end of word; otherwise, returns False.

starts_with(prefix):

Traverses the Trie based on the input prefix. Returns True if all characters of the prefix are found in the Trie; otherwise, returns False.

Real time applications - Auto-autocomplete:

Search Engines:

- As users type a search query, autocomplete provides instant suggestions based on popular searches, user history, and trending topics, accelerating the search process and guiding users to relevant results.

Text Editors and IDEs:

- In programming environments, autocomplete suggests code snippets, variable names, and function calls, reducing typing errors and speeding up development.

Forms and Applications:

- When filling out online forms, autocomplete can pre-fill information like addresses, email addresses, and names based on previously entered data, saving time and ensuring accuracy.

Messaging Apps:

- Autocomplete suggests words and phrases as users type, making communication faster and more convenient, especially on mobile devices.

Content Creation Platforms:

- For bloggers, writers, and content creators, real-time spell check is crucial for producing polished, professional content.

Educational Tools:

- Spell check assists students and language learners in identifying and correcting errors, aiding in the development of accurate writing skills.

Real time applications - Spell Check

Word Processors and Email Clients:

- Real-time spell check highlights misspelled words as they are typed, offering

immediate corrections and improving the quality of written communication.

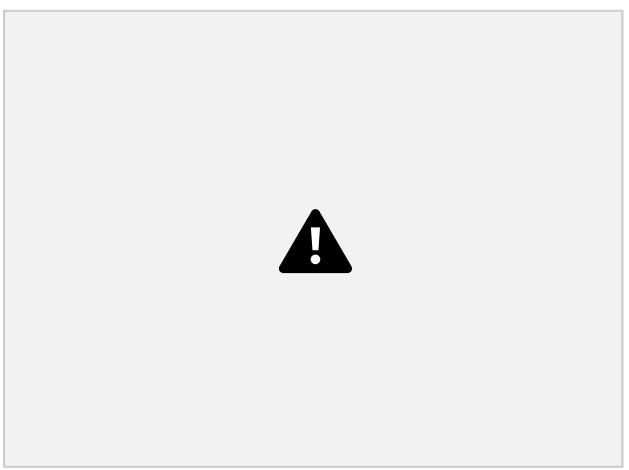
Web Browsers:

- Many browsers include built-in spell checkers that function in text fields on websites, ensuring that online content and messages are free from common errors.

2.10 SEGMENT TREE:

- A Segment Tree is a binary tree where each node represents a segment or range of the original array.
- The root node represents the entire array, and leaf nodes represent individual elements.
- Internal nodes store aggregated information (e.g., sum, minimum, maximum) of their child segments.
- Following are the steps for constructing a segment tree:
    - Start from the leaves of the tree
    - Recursively build the parents from the merge operation

The following diagram shows a segment tree built for an array [1, 4, 5, 9, 10, 12] of size 6



Operations:

- **Build:** Constructs the tree recursively, typically taking O(n) time.
- **Query:** Retrieves aggregated information for a given range in O(log n) time.
- **Update:** Modifies a single element in the array and updates affected nodes in the tree in O(log n) time.

Space Complexity: O(n), as it requires approximately 2n nodes for a complete binary

tree.

- **Advantages:** More versatile for various range query types (e.g., sum, min, max, GCD, etc.) and can be extended to handle 2D queries.
- **Disadvantages:** Can be more complex to implement compared to BITs.

2.11 BINARY INDEXED TREE (FENWICK TREE)

- A Binary Indexed Tree is an array-based data structure that efficiently calculates prefix sums and supports point updates.
- It leverages binary representations of indices to efficiently navigate and update relevant elements.

Operations:

- **Build:** Constructs the BIT, typically taking O(n log n) time by repeatedly calling the update operation.
- **Prefix Sum Query:** Calculates the sum of elements from index 0 to a given index in O(log n) time.
- **Update:** Modifies a single element and updates the BIT in O(log n) time.

Space Complexity: O(n), as it uses an array of size n+1.

Advantages: Simpler to implement and often has a smaller constant factor in its time complexity, making it faster in practice for certain operations (especially prefix sums).

Disadvantages: Primarily designed for prefix sum queries; extending it to other range operations (like minimum or maximum) is less straightforward or requires modifications.

2.11 APPLICATIONS

1. Trees in Databases

- B-Trees and B+ Trees are widely used in database indexing.

Why?

Balanced trees ensure logarithmic search time.

Can handle large datasets stored on disk.

2. Trees in Compilers

- Compilers use syntax trees to represent program structure.

Parse Tree (Concrete Syntax Tree): Represents grammar of code. Abstract Syntax Tree (AST): Simplified hierarchical structure used in semantic analysis and optimization.

Example:

Expression: (a + b) c

AST:

()

/ \

(+) c

/ \

a b

This tree helps the compiler generate machine code step by step.

3. Trees in Dictionaries

Tries (Prefix Trees) are used to store words for fast lookup.

Operations like insert, search, autocomplete are O(length of

word). Example:

Insert words: `cat, car, can`

Trie structure shares common prefix `"ca"`.

2.12 REAL-WORLD PROBLEMS: SPELL CHECKERS, AUTO-SUGGESTION ENGINES, HUFFMAN CODING.

a) Spell Checkers

Spell checkers use Tries to store dictionary words.

When a word is typed:

Search in Trie → if not found → suggest corrections using edit distance + traversal.

b) Auto-Suggestion Engines (Autocomplete)

Uses Tries or Ternary Search Trees.

Example: typing `"ap"` suggests `"apple, application, april"`.

Works by traversing Trie to the `"ap"` node, then listing all words in its subtree.

c) Huffman Coding (Data Compression)

Huffman Coding builds a Binary Tree for prefix-free encoding.

Frequently used characters get shorter codes; rare ones get longer

codes. Example:

Frequencies: `a:5, b:9, c:12, d:13, e:16, f:45`

Huffman Tree assigns codes like:

```
```

f → 0

c → 100

d → 101

a → 1100

b → 1101

e → 111

```

Efficiently reduces file size in compression algorithms (ZIP, JPEG, MP3).
```